The Journal of Systems and Software 234 (2026) 112723

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Applying generative artificial intelligence for vulnerability fixing in a proprietary software ecosystem

Luiz Alexandre Costa [a,*], Awdren Fontão [b], Rodrigo Pereira dos Santos [a], Alexander Serebrenik [c]

[a] Federal University of the State of Rio de Janeiro (UNIRIO), Rio de Janeiro, Brazil
[b] Grosso do Sul (UFMS), Federal University of Mato, Campo Grande, Brazil
[c] (TU/e), Eindhoven University of Technology, Eindhoven, Netherlands

ARTICLE INFO

Dr. Christoph Treude

Keywords:
Software ecosystems
Software quality
Software asset management
Incident management
Generative artificial intelligence

ABSTRACT

Context: Large organizations often operate within proprietary software ecosystems (PSECO), composed of interdependent software artifacts, multiple actors, and centralized governance. In such environments, addressing security vulnerabilities is particularly challenging due to strict quality standards, regulatory constraints, and the risk of cascading failures. These challenges can compromise delivery schedules, increase operational risk, and force teams to interrupt planned work to address urgent issues. **Goal:** In response to these challenges, recent advances in Generative Artificial Intelligence (GenAI), particularly large language models (LLM), have emerged as a promising avenue to support software engineering tasks such as code generation and automated repair. This study investigates how a GenAI-based approach can support vulnerability remediation in PSECO while preserving delivery cadence in Continuous Integration and Continuous Delivery (CI/CD) pipelines. **Methods:** We conducted a participative case study in a large global organization to design and evaluate PSECO-SafePatch, an approach composed of: (i) a structured remediation process tailored to enterprise constraints; and (ii) a web-based tool integrating Fortify static analysis with GenAI-based patch generation. The approach includes human-in-the-loop validation and aligns with DevOps practices. **Results:** PSECO-SafePatch reduced mean remediation time by 84 % with an 89 % patch success rate. It also reduced cognitive overload by guiding developers through structured validation, fostering trust without overreliance. **Conclusion:** The findings show that GenAI-supported remediation is feasible and effective in PSECO. Human-in-the-loop validation preserved critical thinking, addressing concerns about blind automation and knowledge erosion, and reinforcing the value of responsible automation at scale.

1. Introduction

The rapid and continuous evolution of Software Engineering (SE) has intensified the demand for practices that enable agile and frequent delivery of value. This need stems from the increasing reliance of businesses on software solutions and the growing expectation from users for continuous improvements Chen (2017). In this context, Continuous Software Engineering (CSE) practices, such as continuous integration and continuous delivery (CI/CD) have become essential to ensure agility, automation, and stability throughout the software life-cycle Fitzgerald and Stol (2017).

Within this fast-paced development environment, proprietary software ecosystems (PSECO) have emerged as complex structures composed of multiple actors, technologies, and software artifacts managed by a central organization (keystone). For instance, SAP's ecosystem establishes specific partnership terms that regulate how external developers join and leave the ecosystem, ensuring stability and aligning incentives. These ecosystems are increasingly adopted by large enterprises to support scalability and innovation. However, the complexity of interdependent systems in PSECO introduces challenges, particularly when responding to critical incidents such as security vulnerabilities Costa et al. (2021).

When vulnerabilities are identified, they often require immediate action. In PSECO, this urgency forces developers to pause their ongoing tasks to remediate the identified vulnerabilities, disrupting delivery cadence and impacting project planning. A recent study highlighted how such interruptions lead to delays and resource reallocation across organizations Dissanayake et al. (2022). Static analysis tools, such as

* Corresponding author.
E-mail addresses: luiz.costa@edu.unirio.br (L.A. Costa), awdren.fontao@ufms.br (A. Fontão), rps@uniriotec.br (R.P. dos Santos), a.serebrenik@tue.nl (A. Serebrenik).

https://doi.org/10.1016/j.jss.2025.112723
Received 17 October 2024; Received in revised form 15 October 2025; Accepted 1 December 2025
Available online 7 December 2025

Fortify[1], help identify security vulnerabilities. However, in most cases, the fixing process remains manual and time-consuming.

To address this challenge, **Generative Artificial Intelligence (GenAI)** has gained attention for its potential to automate vulnerability remediation Hilario et al. (2024). Among GenAI techniques, a prominent class involves the use of **Large Language Models (LLM)** trained on code and natural language. These **LLM-based approaches** can generate context-aware patches, accelerating the remediation process and reducing developer effort Zubair et al. (2024). However, since PSECO exhibit specific characteristics, such as centralized governance, strong interdependencies between software assets, and strict quality and security requirements, the application of GenAI-based remediation demands careful validation to ensure code quality and stability Cotroneo et al. (2024).

In this scenario, our work investigates how a GenAI-based approach can support vulnerability remediation in PSECO while preserving delivery cadence in CI/CD pipelines through the following main research question (RQ): *How can a GenAI-based approach support vulnerability incident remediation in a PSECO?* To answer this question, we conducted a participative case study in a large global organization operating in the financial domain. The study involved the design, implementation, and evaluation of *PSECO-SafePatch*, a GenAI-based approach that consists of a structured vulnerability remediation process supported by a web-based tool integrating Fortify static analysis results with LLM-based patch generation services.

As contributions, we (i) introduce PSECO-SafePatch, an approach that combines a structured remediation process with a web-based tool to integrate GenAI into CI/CD pipelines within a PSECO; (ii) present an architecture that enables secure, traceable, and human-validated patching workflows; (iii) provide empirical evidence of improvements in productivity, patch quality, and developer perception through a participative case study; and (iv) offer a sociotechnical reflection on the risks of GenAI overreliance, highlighting the role of human oversight in preserving critical thinking and team autonomy.

This article is organized as follows: Section 2 presents background; Section 3 reports the related work; Section 4 outlines the research design and methodological steps; Section 5 describes the case organization; Section 6 details the approach; Section 7 reports the research method and the approach evaluation; Section 8 discusses the findings and implications; Section 9 presents threats to validity and limitations; and finally, Section 10 concludes the work.

2. Background: Challenges faced by proprietary software ecosystems

In this section, we present the background to understand the challenges addressed by our work. Software Ecosystems (SECO) are sociotechnical environments composed of a technological platform, internal and external developers, and multiple stakeholders who collaborate to build and evolve software solutions Bosch (2009). SECO include actors such as suppliers, IT service providers, business partners, and keystone organizations responsible for coordinating and sustaining the ecosystem, as illustrated in Fig. 1.

SECO can be classified into proprietary, open source, and hybrid based on the nature of source code protection and revenue generation models Manikas and Hansen (2013). Our work focuses on PSECO, where software artifacts are protected by confidentiality agreements managed by a central organization (keystone) Manikas (2016).

In recent years, many organizations have evolved from isolated development teams to complex, interconnected networks of interdependencies to support fast innovation and scalable platforms Santos (2016). This transformation increases the complexity of coordination, especially in maintenance tasks such as vulnerability remediation. In PSECO, third-party contributions are regulated by strict policies, which limits collaboration and adds overhead to incident response Outão et al. (2025).

Security vulnerabilities are a critical class of incidents in software ecosystems. They originate from coding flaws, outdated dependencies, or misconfigurations and can cause incidents. According to the Information Technology Infrastructure Library (ITIL), an incident is any unplanned interruption or reduction in the quality of an IT service, including software defects, security vulnerabilities, and system outages. In this context, both faults (such as vulnerabilities) and their resulting failures can initiate incident management processes. These failures may expose systems to data breaches, service outages, or exploitation Doynikova and Kotenko (2016), Makrakis et al. (2021). In addition, the integration of multiple suppliers and systems increases the risk of cascading effects, where a vulnerability in one component can affect interconnected services across the platform.

When a vulnerability is detected, it usually requires urgent attention. However, remediation efforts often demand that developers interrupt ongoing project work, disrupting cadence, and affecting team productivity. In PSECO, these challenges are amplified by limitations in code access across organizational boundaries Manikas (2016). IT managers must balance the need for quick remediation with the goal of maintaining continuous delivery in CI/CD pipelines Fenz et al. (2014), Wu and Wu (2020).

Tools such as Fortify help detect vulnerabilities through static code analysis Lenarduzzi et al. (2019), Vetro (2012), but these tools do not automate the fix process itself. Fig. 2 shows how an SQL Injection vulnerability can expose systems to high-impact threats.

To help address these challenges, Generative Artificial Intelligence (GenAI) has gained relevance in SE for automating tasks such as code generation, optimization, and vulnerability remediation Sherje (2024). In particular, LLM-based approaches, where models trained on extensive codebases generate context-aware patches, have shown promise in reducing manual effort and accelerating the fixing process Ebert and Louridas (2023), Tembhekar et al. (2023).

In the context of PSECO, GenAI may support the continuity and resilience of CI/CD pipelines by offering timely solutions to security issues. However, the use of AI-generated code must be treated cautiously. As noted by Cotroneo et al. (2024), the correctness of these patches must be validated to avoid regressions or new vulnerabilities, especially in sensitive systems managed by legal constraints.

3. Related work

As related work, we identified existing approaches that leverage GenAI for the identification and remediation of security vulnerabilities. We focused on works that automated the fixing process and assisted developers in CI/CD pipelines. Le Goues et al. (2011) proposed a pioneering approach for automatic software repair based on genetic programming techniques called *GenProg*. The method takes as input a faulty program and a test suite containing both passing and failing test cases. *GenProg* demonstrated success in repairing real-world bugs in legacy C programs across a range of fault types, such as buffer overflows, showing the viability of automated repair in production-scale systems.

A more recent approach, Bouzenia et al. (2024), proposes an autonomous framework for automatic program repair using LLM called *RepairAgent*. The method treats the LLM as a reasoning agent capable of orchestrating repair actions by invoking a set of external tools within a defined execution loop. Unlike prompt-based methods, *RepairAgent* employs a finite-state machine to guide interactions and tool usage, demonstrating strong performance in open source benchmarks such as Defects4J and GitBug-Java.

Dakhel et al. (2023) conduct an empirical study evaluating GitHub Copilot on fundamental programming tasks, including algorithms and data structures. The authors analyze the correctness, reproducibility, and similarity of the solutions generated by Copilot, and compare them with human responses on a set of programming tasks. Although Copilot

[1] https://www.opentext.com/products/fortify-static-code-analyzer

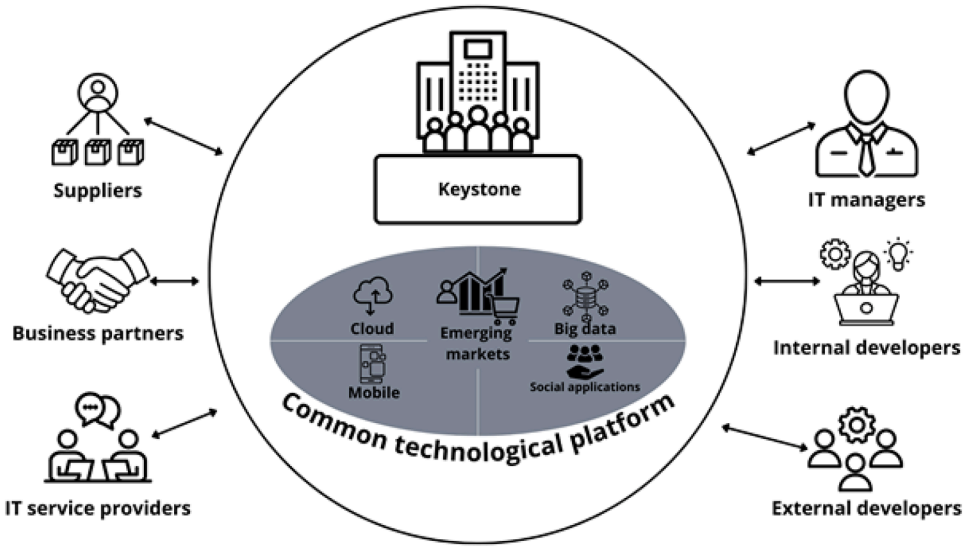

Fig. 1. SECO actors.

```
public boolean login(String username, String password). {}
    String sql =
    "select * from users where username = " + "'" + username + "'" + " and " +
          "password " + " = " + "'" + password + "'";

    ResultSet result = stmt.executeQuery(query);

    if(result.next()) {
        /* Login Success */
        return true;
    }
    else. {
        /* Login Failed */
        return false;
    }
}
```

Fig. 2. Example of vulnerability incident.

is capable of generating functional solutions, the results indicate that these often contain bugs or do not follow the correct algorithmic approach.

To compare our approach with existing solutions in the literature and industry, we identified a set of relevant features and capabilities based on synthesized adoption dimensions discussed in recent research on the use of GenAI in SE Russo (2024). We selected the dimensions that are particularly salient for vulnerability remediation in PSECO, which are environments characterized by centralized governance, strict control over software artifacts, limited openness to externl tools or code sharing, and rigorous compliance with security and confidentiality policies Outão et al. (2025). These characteristics impose specific constraints on the use of AI-generated patches, especially regarding traceability and integration with internal CI/CD workflows. Table 1 presents a comparative overview of selected features across related approaches, including ours. The next section presents the research design that guided the development and evaluation of our approach.

4. Research design

To guide the structure of our work, we adopted a conceptual research decision-making structure to support the creation of a coherent research design and the selection of appropriate research methods Wohlin and Aurum (2015), Wohlin and Runeson (2021). Inspired by established methods in Empirical Software Engineering (ESE) Wohlin et al. (2012), this structure allowed us to align our methodological choices with the objectives and context of the study.

Accordingly, our research process was organized into three sequential steps: (i) diagnosing the current state-of-practice in the case organization; (ii) designing and developing the PSECO-SafePatch approach; and (iii) evaluating this approach through a participative case study. This stepwise design ensured a systematic transition from problem understanding to solution design and empirical evaluation. In the following, we detail each step and indicate their connection to the structure of this article, as illustrated in Fig. 3:

- **Step 1: Diagnose the State-of-Practice.** To understand the current vulnerability incident remediation process within the organization, we adopted an exploratory approach combining contextual inquiry with practitioners, informal observations, process mapping, and internal document analysis. These activities, detailed in Section 5, allowed us to diagnose limitations and bottlenecks, while identifying requirements for a more robust remediation approach. This step corresponds to the initial stage of problem identification, as typically adopted in empirical investigations involving real-world software practices Wohlin and Runeson (2021);
- **Step 2: Design and Develop the Approach.** Guided by outputs from Step 1, we followed an engineering method to design and develop the PSECO-SafePatch approach. This method is characterized by the study of current solutions, the proposal of changes, and sub-

Table 1
Updated comparative of GenAI approaches, where: fx1 (fully supported); fx2 (partially supported); and fx3 (not supported).

Feature/Capability	GenProg	RepairAgent	GitHub Copilot	PSECO-SafePatch
Workflow Integration Compatibility	fx3 No integration focus	fx3 Prototype-stage	fx2 IDE-focused, limited to developer tools	fx1 Native CI/CD (Fortify + DevOps tools)
Security/Privacy Constraints	fx3 Open source test scenarios	fx3 Online public API	fx3 Uses public cloud LLM, lacks enterprise compliance	fx1 Azure OpenAI private instance
Complementary Tool (human-in-the-loop)	fx3 Fully automatic genetic repair	fx3 Autonomous LLM agent	fx2 Optional editing, but mostly passive use	fx1 Mandatory human validation step
Task Complexity	fx2 Tested on isolated defects	fx2 Limited to prompt-based scenarios	fx3 General code assistance, not vulnerability-specific	fx1 Designed for enterprise-scale, interdependent systems
Code Improvement and Maintenance	fx1 Test-driven code patching	fx1 Focuses on bug repair with LLM reasoning	fx3 Aims at productivity, not fix validation	fx1 Patch quality tracked, reusable fix templates
Efficiency and Automation	fx1 Genetic algorithm automates fix generation	fx3 Not benchmarked	fx2 No focus on automation pipeline, just assistive	fx1 84 % productivity gain in case study
Trust and Reliability	fx3 No human validation or oversight	fx3 No process oversight or reporting	fx3 No guarantees on patch quality or traceability	fx1 Human-in-the-loop validation

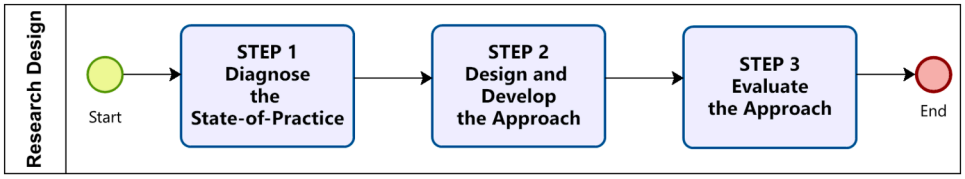

Fig. 3. Research design.

sequent evaluation in context Wohlin et al. (2012). The approach integrated process innovation with tool implementation, combining GenAI capabilities, human-in-the-loop validation, and compliance mechanisms. Section 6 describes the solution developed in close collaboration with practitioners within the organization; and

- **Step 3: Evaluate the Approach.** In the final step, we conducted a participative case study to evaluate both the efficiency and effectiveness of the PSECO-SafePatch approach, as well as the perceptions of IT managers and developers regarding its usefulness and ease of use. This method involved the guidelines proposed by Runeson and Höst (2009). We employed both quantitative and qualitative methods, including metrics analysis and semi-structured interviews with IT managers and developers, to assess the outcomes. Section 7 presents the evaluation design and results.

In the following section, we analyze the organization's current software development and vulnerability incident remediation processes to identify challenges and opportunities for improvement.

5. Case description

Due to privacy reasons, the organization's name is omitted. From now on, we refer to it as organization "X". Founded over 80 years ago, the organization "X" is one of the largest insurance groups in Latin America, operating internationally in multiple segments, including auto, property, health, capitalization, and open supplementary pension. The organization maintains more than 200 branches (service centers, offices, and customer service units) throughout the country and partners with more than 40,000 insurance brokers. Although geographically distributed, all software development initiatives are governed centrally by the IT department of the holding company, which coordinates projects and infrastructure across business units through a unified governance model.

In a highly competitive business environment, IT plays a critical role in sustaining the organization's performance by providing information flows that add value without compromising operational efficiency Rugeviciute et al. (2023). As part of a broader digital transformation (DT) initiative, organization "X" has shifted its strategic focus toward client-centered services and business value generation. DT involves leveraging digital technologies to solve traditional organizational problems, such as drops in productivity, performance, and agility, and requires structural changes within companies Vial (2021).

A major challenge facing the organization "X" is that the business rules of the predigital era are no longer sufficient. As part of this modernization, the organization adopted the DEVOPS culture, which combines practices and tools to increase agility in software delivery compared to traditional development models Davis and Daniels (2016). However, the increasing market pressure for rapid delivery has resulted in organizational strain, leading to problems such as late projects, budget overruns, developer turnover, employee health concerns, and service disruptions caused by deployments Govindaras et al. (2023), Romano et al. (2024).

To address these challenges, in the first semester of 2019, organization "X" implemented change and incident management practices aligned with the ITIL (Information Technology Infrastructure Library) framework[2] As part of this framework, security vulnerability incidents are managed by a specialized department known as Information Security. Once vulnerabilities are detected, they are prioritized and immediately assigned to the responsible development teams.

Upon receiving a vulnerability correction task, development teams must pause their ongoing activities to prioritize remediation. This dynamic affects project timelines and delivery schedules, as security issues demand immediate attention. The remediation process is closely coordinated with the Information Security department to ensure that all fixes fully mitigate the identified threats.

In addition, the IT Board is notified when a vulnerability is identified. The Information Security department ensures that upper management is informed and initiates a broader incident review to proactively identify similar vulnerabilities across the IT infrastructure. A dedicated project management professional (PMP) monitors the entire remediation process, provides regular updates to IT Board, and ensures alignment with organizational risk management protocols.

In summary, the organization "X" is undergoing a digital transformation by integrating DEVOPS practices and ITIL guidelines while maintaining a strong focus on security through its Information Security department. The urgency imposed by vulnerability remediation and the growing demand for automation in CI/CD environments have exposed

[2] ITIL is a set of practices for IT service management aimed at organizing processes and improving efficiency Agutter (2020).

gaps in the current process (“ as is” process). These challenges highlight the need for a new structured approach that integrates GenAI into the existing CI/CD pipeline to support secure and efficient vulnerability remediation.

6. The PSECO-Safepatch approach

In this section, we present the PSECO-SafePatch approach developed to address the challenges of vulnerability remediation within a PSECO. The approach consists of two complementary elements: (i) a structured vulnerability remediation process designed to integrate GenAI capabilities into the existing CI/CD pipeline, and (ii) a web-based tool specifically developed to instantiate critical parts of this process, integrating Fortify static analysis results with LLM-based patch generation services.

The following subsections present: (i) the motivation for developing the approach (Section 6.1); (ii) the redesigned vulnerability remediation process, also referred to as the “ to be” process (Section 6.2); (iii) the overall architecture of the PSECO-SafePatch tool (Section 6.3); and (iv) its main functionalities and integration features (Section 6.4).

6.1. Motivations for the approach

The development of the PSECO-SafePatch tool was driven by several needs identified in the context of PSECO. Based on the investigation of the “as is” process and the iterative refinement cycles with stakeholders, we identified three recurring pain points that became central motivations for designing the PSECO-SafePatch tool. These stakeholders included 18 developers and technical leaders involved in the organization’s vulnerability remediation process. These motivations emerged as factors that directly influenced the requirements and goals of the solution. Although other observations were made during the case study, the following three stood out due to their frequency, strategic impact, and direct connection to process inefficiencies:

- **Automating Manual Analysis.** Manual vulnerability analysis is time-consuming and prone to errors Austin et al. (2013), especially when developers need to assess side effects across multiple components. This challenge is exacerbated in PSECO, where systems can be composed of tightly coupled components maintained by different actors under centralized governance mechanisms Alves et al. (2017). The complexity of PSECO lies in high interdependency, coordination across contractual boundaries, and the need to preserve ecosystem stability when changes occur Outão et al. (2025). In such environments, even a small code fix can produce ripple effects throughout the ecosystem. The PSECO-SafePatch approach can automate the generation of context-aware patches, reduce the reliance on manual inspection, and enable faster, safer vulnerability remediation.
- **Reducing Disruption to Developer Workflow.** During the mapping of the “as is” process, we observed that developers often had to interrupt their ongoing activities to address vulnerabilities. This interruption caused project delays and decreased productivity. Similar findings were reported by Dissanayake et al. (2022), who observed that patching activities frequently disrupt ongoing development tasks, leading to delays and reduced productivity, particularly when developers are forced to switch context to address urgent vulnerabilities. Thus, we identified the need for a solution that would address vulnerabilities without distracting developers from their main tasks, which motivated the development of the PSECO-SafePatch tool; and
- **Optimizing Cost-Effectiveness.** Through the iterative refinement cycles, we identified opportunities to improve the efficiency of vulnerability fixing by optimizing resource usage and reducing the time spent remediating issues. Similar concerns have been addressed in recent research. Tian et al. (2025) proposed a cost-efficient vulnerability detection approach using cross-modal adversarial reprogramming, demonstrating that automating parts of the remediation pipeline can significantly reduce manual effort and operational costs. These insights reinforced our motivation to develop the PSECO-SafePatch tool, which aims to reduce overhead by automating critical steps in the fixing process.

6.2. Proposed vulnerability remediation process

We proposed a restructured (“to be” process) CI/CD pipeline to streamline the identification, prioritization, and remediation of security vulnerabilities, as illustrated in Fig. 4. This new process was designed not only to modify operational workflows but also to be supported by a technological solution tailored to the needs of PSECO environments.

To this end, we developed the PSECO-SafePatch approach, which combines a structured vulnerability remediation process with a web-based tool that integrates Fortify static analysis outputs and LLM-based patch generation services. The following list presents a detailed view of the proposed (“to be”) process and is provided for transparency and traceability. It illustrates how GenAI-based remediation activities are systematically incorporated into the CI/CD pipeline.

1. **Integrate Source Code Applications.** The process starts with Jenkins, which integrates with the source code repositories to continuously monitor and manage the codebase. Jenkins automates the initial stages of CI/CD pipeline, ensuring that the latest code is always analyzed for potential vulnerabilities. By continuously integrating changes from multiple developers, Jenkins sets the stage for a transition to the vulnerability detection phase, ensuring that the most recent codebase is always subjected to security analysis.

   This step should ensure that security analysis reflects the current state of the codebase. In PSECO, governance mechanisms must enforce strict control over proprietary artifacts Outão et al. (2025). Automated integration helps maintain this control, avoid security blind spots, and support the keystone’s responsibility for ensuring platform integrity;
2. **Analyze Detected Vulnerabilities.** After Jenkins integrates the source code, Fortify Static Code Analyzer performs a comprehensive security scan. It generates a report on the vulnerabilities present in the code, including details about the type, severity, and specific location of each issue. This list serves as the foundation for the next steps, guiding the development team and GenAI engine in prioritizing and addressing the most critical vulnerabilities first.

   In PSECO, this step supports the keystone’s responsibility for platform security and integrity Outão et al. (2025). Moreover, early detection reinforces control over proprietary software artifacts and helps prevent the spread of vulnerabilities across dependent systems. Table 2 reproduces an example of a detailed report of security issues categorized by Fortify, showing the criticality of each one;
3. **Search for Fix Scripts.** Once the vulnerabilities are identified, PSECO-SafePatch tool searches a local knowledge base for existing fix scripts. If a matching script is found, it is applied to the code. The knowledge base functions as a cache of previously approved fixes, reducing the need for repeated GenAI processing. By leveraging previously validated fixes, the tool can address recurring vulnerabilities without incurring additional costs associated with frequent API calls to the OpenAI service. If no matching script is found, PSECO-SafePatch tool uses an GenAI engine to generate a custom fix for the detected vulnerability.

   Due to contractual agreements, OpenAI cannot be trained on public data. Therefore, the organization has subscribed to a private cloud service with Microsoft, ensuring that the data used for generating fixes is not available for public AI training. This organization strategy aligns with the characteristics of PSECO, where maintaining the confidentiality and integrity of sensitive data is paramount. The private subscription guarantees that sensitive data remains secure and isolated from public AI models.

   This step reflects core PSECO characteristics, such as strict control over intellectual property and restricted development processes

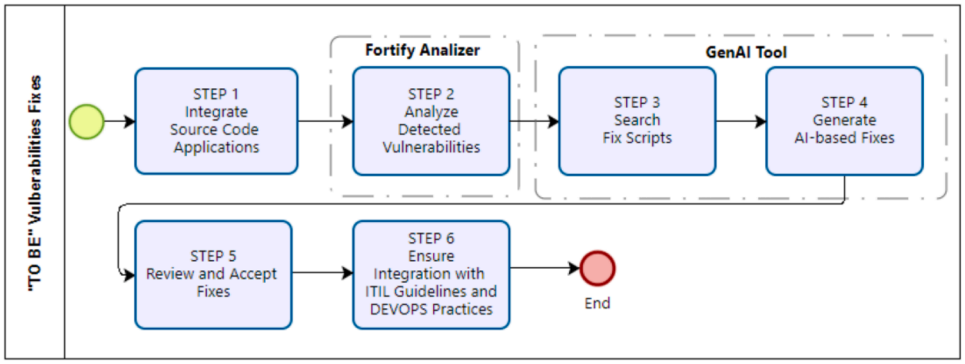

Fig. 4. Proposed ("to be" process) CI/CD pipeline relating to vulnerability fix.

Table 2
Example of issue breakdown by Fortify categories, where C-Critical; H-High; M-Medium; and L-Low.

Category	C	H	M	L	Total
Byte Array to String Conversion	0	0	0	2	2
Comparison of Boxed Primitive Types	0	0	0	6	6
Constructor Invokes Overridable Function	0	0	0	8	8
Non-Synchronized Method Overrides Synchronized Method	0	0	0	1	1
String Comparison of Float	0	0	0	2	2
Cross-Site Request Forgery	5	0	0	0	5
Denial of Service	1	0	0	0	1
Denial of Service: StringBuilder	0	10	0	0	10
Hardcoded Domain in HTML	0	0	18	0	18
J2EE Misconfiguration: Incomplete Error Handling	0	2	0	0	2
Missing XML Validation	0	10	0	0	10
Poor Error Handling: Overly Broad Catch	0	0	1	0	1
Poor Style: Non-final Public Static Field	0	0	0	2	2
Race Condition: Format Flaw	1	0	0	0	1
SQL Injection	3	0	0	0	3
Struts: Form Does Not Extend Validation Class	0	18	0	0	18
Struts: Plugin Framework Not In Use	0	0	0	1	1
Struts: Unused Action Form	0	0	0	2	2
Struts: Validator Turned Off	0	0	0	2	2
System Information Leak: HTML Comment in JSP	2	0	0	0	2
System Information Leak: Internal	1	0	0	0	1
Unchecked Return Value	0	0	1	0	1

Outão et al. (2025). The reuse of approved fixes preserves platform integrity, while private infrastructure supports governance practices for confidentiality and manages technological variability;

4. **Generate AI-based Fixes.** The GenAI engine running on an exclusive Azure OpenAI tenant, returns suggestions and automated corrections for the code. This step involves using advanced AI capabilities to generate precise and context-aware fixes for the vulnerabilities. Based on the example mentioned in Table 2, Fig. 5 shows a *SQL Injection* vulnerability in the first part, where user input is directly concatenated into a SQL query. The second part illustrates the fix generated by the PSECO-SafePatch tool, which involves using a *prepared statement* with parameterized queries to securely handle user input, thereby preventing the *SQL Injection* attack.

   This step aligns with the PSECO principle of ensuring platform security and reliability. By automating fixes within a controlled environment, the keystone enforces compliance and reduces the risk of manual errors, consistent with its governance role Outão et al. (2025). Using a secure, isolated infrastructure also preserves data integrity and supports confidentiality standards;
5. **Review and Accept Fixes.** This step involves a human-in-the-loop prompt refinement and validation process, where the development team reviews the AI-generated fixes. Developers assess whether the suggested corrections are appropriate for the specific code context and may choose to accept, modify, or reject the proposals based on their expertise and the application's requirements. This human oversight ensures that only the most suitable corrections are integrated into the codebase, preserving both security and functionality while maintaining developer control over the final implementation.

   In PSECO, this review is essential to preserve control over code changes and ensure that all fixes comply with internal quality and intellectual property standards Outão et al. (2025). Human oversight reinforces governance practices that regulate contributions and maintain code integrity; and
6. **Ensure Integration with ITIL Guidelines and DEVOPS Practices.** Throughout the entire process, PSECO-SafePatch approach aligns with ITIL guidelines and incorporates DEVOPS practices to ensure that vulnerability management is systematic. ITIL guidelines are followed to manage changes and incidents, providing a structured method to handling the integration of security fixes. Meanwhile, DEVOPS practices ensure CI/CD, facilitating the rapid and reliable delivery of updates.

   This step aligns with the core PSECO characteristic of maintaining platform stability and scalability. Following standardized frameworks ensures consistent asset management and supports ecosystem health through traceable and auditable processes Outão et al. (2025).

6.3. Architecture of the PSECO-Safepatch tool

The tool is a web-based application designed to automate, track, and accelerate vulnerability remediation within a PSECO by integrating DevOps and ITIL practices with GenAI. It follows a three-tier architecture composed of client, server, and database layers, implemented with **React.js** on the front end, **NestJS** on the back-end, and **PostgreSQL** as the persistent data store.

The application is embedded in a CI/CD pipeline, where its operation begins when a commit is made to the code repository. This event triggers the CI pipeline, which automatically retrieves the source code of an J2EE application and performs a static security analysis using Fortify, generating a `.sarif` report listing detected vulnerabilities.

This report is sent to the application back-end, which first queries its local knowledge base for previously validated fix scripts. If a match is not found, the application constructs a structured prompt based on the `.sarif` content and the original code, and sends it to a GenAI model hosted on a private *Azure OpenAI* instance, in compliance with the security and confidentiality standards required in regulated enterprise environments.

The response generated by the LLM is parsed and presented to the developer through an interactive *code diff* interface, highlighting removed, changed, and added lines. This review screen is part of a **human-in-the-loop validation** step, where developers and security reviewers assess the suitability of the proposed patch. The fix can be accepted, modified, or rejected based on best practices, architectural constraints, and functional impact.

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query = "SELECT * FROM items WHERE owner = '"
                        + userName + "' AND itemname = '"
                        + itemName + "'";
ResultSet rs = stmt.execute(query);
```

```
String userName = ctx.getAuthenticatedUserName();
String itemName = request.getParameter("itemName");
String query =
        "SELECT * FROM items WHERE itemname=? AND owner=?";
PreparedStatement stmt = conn.prepareStatement(query);
stmt.setString(1, itemName);
stmt.setString(2, userName);
ResultSet results = stmt.execute();
```

Fig. 5. SQL Injection fix generated by PSECO-SafePatch tool.

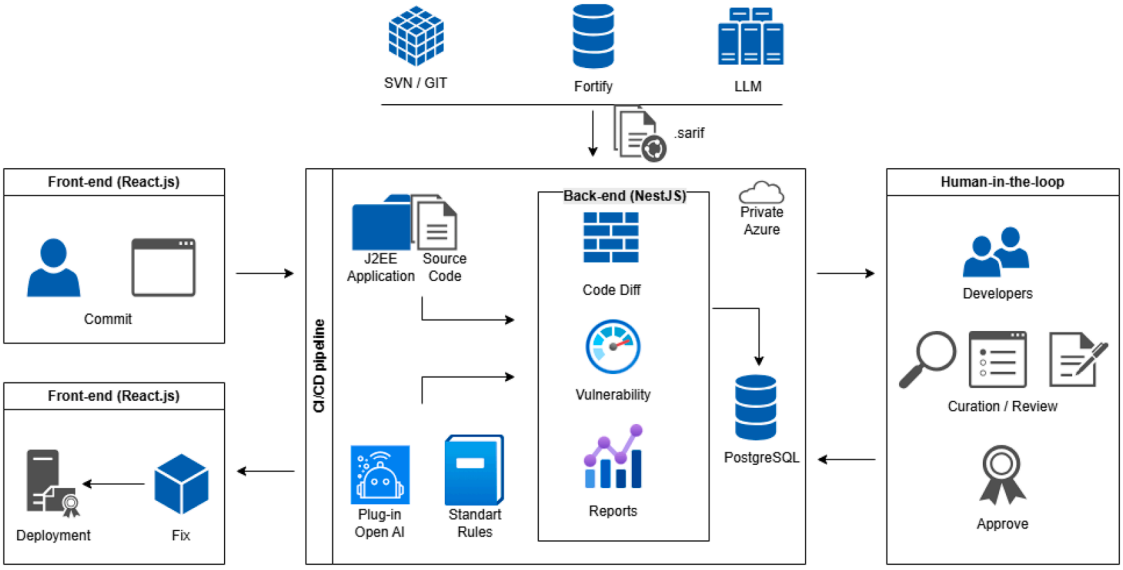

Fig. 6. C4 Architecture Model at the container level.

Once validated, the fix is stored in the local history for reuse in future occurrences. The updated code is then pushed back into the CI/CD pipeline, passing through the usual build and deployment stages until it is deployed to the appropriate environments. This end-to-end integration ensures a fluid, traceable, and efficient workflow for vulnerability detection, remediation, and delivery. Beyond reducing developer cognitive load, the tool enhances security governance and compliance across complex enterprise applications.

Fig. 6 presents the C4-Architecture Model[3] at the container level for the tool, illustrating how the core elements of the solution are organized and interact within the CI/CD pipeline. This view highlights the main containers involved in the architecture, such as the continuous integration process, the GenAI-based correction module, and the human-in-the-loop curation flow. External systems such as source control (SVN/Git), static analysis tool (Fortify), and the LLM engine are shown integrated with internal components responsible for source processing, rule application, vulnerability evaluation, and report generation. The presence of structured entry and exit points (e.g., commit and deployment) further reinforces the automated and traceable nature of the pipeline.

[3] https://c4model.com/

6.4. Functionality of the PSECO-Safepatch tool

The PSECO-SafePatch tool is a web application developed with a call to OpenAI API service designed to automate the remediation of code vulnerabilities. Using GenAI capabilities derived from the OpenAI API, PSECO-SafePatch tool integrates into CI/CD pipelines, identifying, prioritizing, and remediating security vulnerabilities. The tool also helps manage time, preventing developers from interrupting their regular activities to fix vulnerabilities.

The functionalities of the PSECO-SafePatch tool were not defined arbitrarily. They were derived from a combination of empirical evidence gathered during the participative case study and structured research planning. Throughout the text, they are referenced as “DRx”, where “DR” stands for Derivation Rationale and “x” is a unique identifier, as follows:

- **DR1**: features such as reducing remediation time and minimizing developer context-switching, identified as challenges in the organization’s vulnerability fixing process;
- **DR2**: limitations observed in the “as is” process, such as manual tracking, unstructured decision-making, and lack of integration with security tools;

Table 3
Mapping between derived functionalities (DF) and derivation rationales (DR).

Code	Functionality	Derived from (DRx)
DF1	Upload of `.sarif` file	DR2, DR4
DF2	Upload of `.ear` file	DR2
DF3	Wizard interface (step-by-step flow)	DR2, DR1
DF4	*Start Fix* button	DR1
DF5	Criticality-based vulnerability list	DR2, DR3
DF6	GenAI-based code fix generation	DR1, DR3
DF7	Side-by-side code comparison	DR3, DR4
DF8	Human-in-the-loop validation	DR3, DR4
DF9	Use of private Azure tenant	DR3

- **DR3**: core characteristics of PSECO (e.g., control over proprietary artifacts, scalability, and security) as identified by Outão et al. (2025); and
- **DR4**: adherence to DevSecOps and CI/CD practices (e.g., .sarif file format, human-in-the-loop validation, and visual code comparison) in proprietary environments.

Table 3 presents the mapping between each functionality (DFx) and its supporting derivation rationales (DRx), providing traceability between the tool's features and the contextual needs that motivated them.

The tool was implemented with Node.js and React JS. The prototype contains approximately 9257 lines of code (LOC), considering only the source files and configuration scripts, excluding third-party dependencies and build artifacts. The backend was developed using NestJS, a progressive Node.js framework that organizes code into modular components such as controllers, services, and modules, following the principles of dependency injection and separation of concerns. In addition, three new data model tables were created, managed by the API through a PostgreSQL connection library. The prototype source code is available in our supplementary material at https://doi.org/10.5281/zenodo.15386005.

We also have a brief description of the tool and its benefits, highlighting how it can integrate into CI/CD pipelines to speed up vulnerability fix. A button highlighted with the phrase *Start Fix* **(DF4)** will take the developer directly to the vulnerability fix process, guiding them through a wizard **(DF3)**.

On the first screen of the PSECO-SafePatch tool setup wizard, the developer is presented with two main options to start the vulnerability analysis and fix process. The first option allows the user to upload a .ear file **(DF2)** to scan source code for vulnerabilities. The second option is to upload a .sarif **(DF1)** (Static Analysis Results Interchange Format) file.

The .sarif file is a standard file format developed by OASIS (Organization for the Advancement of Structured Information Standards) to store the results of static code analysis. It is designed to facilitate interoperability between different security tools, allowing the results to be shared among platforms Anderson et al. (2019). In our context, this file is generated by Fortify Static Code Analyzer when it performs a security analysis of a code, providing a report and a .sarif files. This file contains structured details about the vulnerabilities found, as shown in Fig. 7.

On the second screen of the PSECO-SafePatch tool setup wizard, the developer sees a list of identified vulnerabilities, organized by criticality levels: critical, high, medium and low **(DF5)**. Each category is presented with a corresponding colored bar and a number next to it indicating the number of vulnerabilities found at that criticality. There is an expand/collapse button associated with each category, allowing the developer to view specific details of the vulnerabilities within each group. The list is loaded from the previously imported .sarif file, which allows the tool to organize and display the vulnerabilities in a clear manner, facilitating the prioritization of fixes.

Once the user has selected a specific vulnerability, the code related to the selected vulnerability is submitted for analysis by the GenAI **(DF6)**. We use OpenAI technology running on the organization's private cloud in Microsoft Azure[4] **(DF9)**. The result is displayed on the third screen, which the user sees a side-by-side comparison between the original code and the code recommendation generated by the GenAI **(DF7)**, as shown in Fig. 8. The visualization follows the traditional model of code comparison tools, highlighting with different colors the removed, changed, and included parts. This layout makes it easy to review the suggested changes, allowing the developer to quickly understand what was adjusted and accept or refine the recommendations before applying them to the base code **(DF8)**.

To integrate GenAI into the PSECO-SafePatch tool, we implemented a call to the OpenAI API service. This service allows for the generation of automated suggestions for vulnerability fixes. The API is consumed by sending structured prompts, also known as prompt engineering, to ensure AI-generated responses are tailored to the specific context of vulnerability fixing.

Prompt engineering is the process of refining prompts (specific instructions given to LLM) to achieve desired outcomes Wang et al. (2023). It involves designing the input text to guide the GenAI in producing relevant and useful responses. This technique is essential for ensuring that the AI's output aligns with the user's needs, especially in complex tasks such as vulnerability analysis, content generation, or decision-making Korzynski et al. (2023).

Fig. 9 illustrates the call to the OpenAI API. First, the OpenAI API key is carried securely without being hardcoded into the code. This key is a unique access key that serves as a credential to authenticate and authorize your requests when using OpenAI services. Next, the prompt engineering is configured through specific parameters. These parameters below are used for controlling the behavior and output of the LLM during response generation. They were selected based on preliminary tests, ensuring that the GenAI responses were adequate to our context.

- **Provide:** "gpt". Specifies the model provider;
- **Model-Name:** "gpt-4o-mini". Defines the name of the model to be used;
- **Temperature:** "1". Controls the randomness of the generated responses. A higher value (closer to 1) makes the responses more creative and varied, while lower values (closer to 0) generate more deterministic and conservative outputs;
- **Top-p:** "1". Determines the cumulative probability threshold for the model's word choices (also known as "nucleus sampling). With a value of 1, the model considers all possible options, while lower values restrict the choice to higher probability words; and
- **Max-tokens:** "4095". Sets the maximum number of tokens (which can be words or parts of words) in the generated response.

In addition, we have the **prompt** parameter. This is the text input that you provide to the model, which acts as the context or question that the model will respond to. Our prompt approach was modeled based on the catalog of Prompt Patterns White et al. (2023), Leão et al. (2024) and OpenAI[5] references. We refined and tested iteratively based on feedback from initial outputs until we reached its current form, as follows:

Each sentence of the prompt is explained below, including the prompt pattern used:

"You are a software security expert focused on vulnerability analysis in Java code."

Purpose: this sentence defines the role that the GenAI should assume. It must act as a software security expert, with a focus on vulnerability analysis in Java code. It contextualizes the model so that it provides appropriate and specific responses to the task. **The prompt pattern used was Persona.** The Persona pattern aims to incorporate a role into the LLM, causing it to assume (or at least attempt to assume) the line of thinking of the specified role.

[4] https://learn.microsoft.com/en-us/azure/ai-services/openai/overview
[5] https://platform.openai.com/docs/guides/prompt-engineering

Fig. 7. Example of.sarif file. The *locations* attribute details where the vulnerability was found in the code, including name and line. Due to privacy concerns, it has been hidden.

Fig. 8. Third step of the setup wizard to code comparison.

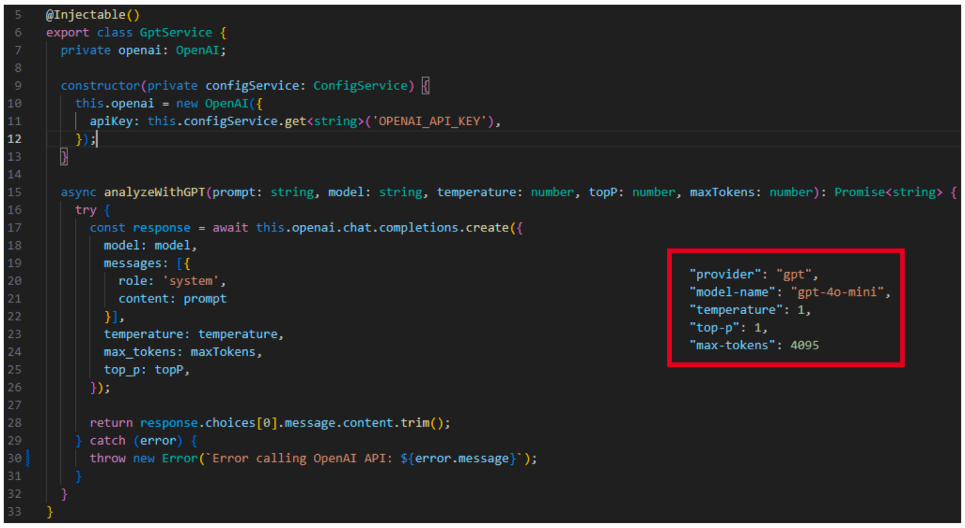

Fig. 9. OpenAI API service parameters.

```
‘‘You are a software security expert focused on
vulnerability analysis in Java code. Your task is
to analyze the attached .ear file and the associated
.sarif report. Focus specifically on the identified
security vulnerability: $vulnerabilitySelected.
Consider security best practices and provide a
detailed analysis, identifying the specific issues
and suggesting appropriate fixes to mitigate security
risks.
The .ear file content is: $earFileContent.
The .sarif file content is: $sarifFileContent.’’
```

“Your task is to analyze the attached .ear file and the associated .sarif report.”

Purpose: the prompt informs the GenAI of the specific tasks to analyze: i) an .ear file, which is a Java EE archive file that may contain web applications and related components; and ii) a .sarif report, which contains details about vulnerabilities identified through static code analysis. **The prompt pattern used was Recipe.** The Recipe pattern is used to obtain a sequence of steps that lead to a desired outcome.

“Focus specifically on the identified security vulnerability: $vulnerabilitySelected.”

Purpose: this sentence directs the GenAI’s attention to a specific vulnerability, whose name or description is passed as a variable (`vulnerabilitySelected`). This ensures that the analysis focuses on the area of greatest interest or concern to the developer. **We used the Context Manager as the prompt pattern.** The Context Manager pattern is used to set the context for the GenAI’s output.

“Consider security best practices and provide a detailed analysis, identifying the specific issues and suggesting appropriate fixes to mitigate security risks.”

Purpose: this sentence instructs the GenAI to base its analysis on security best practices. It should provide a detailed analysis, identify specific issues related to the highlighted vulnerability, and suggest fixes that can mitigate security risks. We should ensure that the GenAI’s response is useful and practically applicable. **The prompt pattern used was Output Automater.** The Output Automater pattern is relevant here because it is asking the GenAI to automate the process of generating an analysis that follows security best practices, which involves creating a structured output.

“The .ear file content is: $earFileContent. The .sarif file content is: $sarifFileContent.”

Purpose: In the final sentence, the content of the .ear and .sarif files is passed directly in the prompt. The variables `$earFileContent` and `$sarifFileContent` represent the content of the respective files. Including the content of these files in the prompt allows the GenAI to have the necessary context to perform an accurate analysis and provide relevant recommendations. **We used the Template as the prompt pattern.** The Template pattern is used to structure the GenAI’s output based on a predefined format, which applies to this case where file content is inserted into the context of the prompt.

7. Case study

We conducted and reported this case study following the guidelines proposed by Runeson and Höst (2009). These guidelines support investigations that seek to generate new insights, ideas, and hypotheses in contexts with limited prior knowledge, while ensuring transparency, rigor, and relevance throughout both the design and evaluation phases.

In our work, the PSECO-SafePatch approach, which comprises both the vulnerability remediation process and its supporting tool, was developed internally within organization “X”. Next, a case study was conducted in the same organization to evaluate how this GenAI-based approach performs in practice.

The purpose of our case study is aligned with the main RQ, which investigates how a GenAI-based tool can support vulnerability incident remediation in a regulated environment, such as PSECO. To deepen this investigation, we formulate two specific subquestions (SQ): SQ1 focuses on efficiency and effectiveness, while SQ2 explores the perceptions of IT managers and developers involved in the remediation process. More details are presented in Section 7.3.

7.1. Case and unit of analysis

In this study, the **case analyzed** was the software development process and the life-cycle of software assets within organization “X”, focusing on the interdependencies involved during the vulnerability remediation process. Given the complexity of PSECO Outão et al. (2025), understanding the dynamics of asset evolution and fix propagation was essential for the investigation.

As the **unit of analysis**, we considered the technological platform of organization “X”, which comprises a vast repository of approximately 2000 Java 2 Platform, Enterprise Edition (J2EE) applications. Each application consists of multiple *.java* files, which are compiled and packaged into structures such as Enterprise Archives (EAR), Web Application Archives (WAR), and Java Archives (JAR), encapsulating business modules and web services. By focusing on the platform as a whole, we were able to observe how its interdependent structure influences the vulnerability remediation process within a PSECO context.

Understanding the software development process and its associated CI/CD pipelines is crucial, as these pipelines serve as the backbone for delivering rapid and reliable software updates. By analyzing vulnerability fixing within this context, we aimed to capture the practical challenges and opportunities for GenAI-based support in continuous delivery environments.

7.2. Rationale for choosing a genAI-based remediation approach

Recent research highlighted a growing movement toward the practical application of LLM in SE tasks, such as automated code repair, test generation, and incident resolution Ahmed et al. (2025). These emerging approaches emphasize the potential of GenAI to development workflows and enhance productivity, particularly when integrated into CI/CD pipelines. However, most existing efforts target general-purpose contexts that may neglect constraints of PSECO.

In PSECO environments, where software artifacts are protected by confidentiality agreements and delivery flows are tightly coupled to compliance and risk controls Outão et al. (2025), integrating GenAI-based tools into automated pipelines requires tailored solutions. Moreover, the interdependencies between systems and the criticality of uninterrupted deliveries further amplify the need for validated, traceable, and secure remediation processes. Given the case study described in our work, we identified the design of a structured GenAI-approach, compatible with CI/CD workflows. Our approach is aligned with the governance and operational needs of PSECO as a promising strategy to improve vulnerability management without compromising delivery cadence or platform stability.

However, the casual adoption of GenAI tools, such as public LLM (e.g., ChatGPT), is insufficient to meet the rigorous demands of enterprise vulnerability remediation Cotroneo et al. (2024). While GenAI can support code generation and automation, concerns about privacy, control, and code correctness persist Dakhel et al. (2023). Similarly, the naive adoption of LLM without tailored processes can lead to low-quality or insecure outputs Tian et al. (2023), which is particularly problematic in environments where vulnerabilities can have cascading impacts across ecosystems (e.g., CrowdStrike incident, where a faulty update caused a massive disruption in approximately 8.5 million systems running Microsoft Windows Chawdhury (2024)). Therefore, generic appli-

cation of LLM cannot fully address the specific operational and governance needs present in PSECO environments.

Finally, based on the practices identified in our previous rapid review (RR) study Costa et al. (2024a), such as: i) dynamic incident response planning; ii) real-time monitoring and prioritization; iii) automation of vulnerability remediation activities; iv) continuous maintenance of a security knowledge base; and v) change management support, our approach focuses precisely on structuring the vulnerability remediation process to maximize the effectiveness of GenAI application. In our case study, we propose and evaluate *PSECO-SafePatch*, a GenAI-based approach that integrates Fortify static analysis results with LLM-based patch generation services, designed specifically to operate within the constraints and requirements of PSECO environments.

7.3. Research question and Study's goal

The goal of our work was structured following the GQM (Goal-Question-Metric) paradigm Basili (1992): **investigate** how a GenAI-based approach can support the remediation of vulnerability incidents **with the purpose of** improving the vulnerability management process **with respect to** efficiency and effectiveness **from the perspective of** IT management and development teams **in the context of** PSECO of a large global organization.

To address this goal, the **main RQ** for our work is: *"How can a GenAI-based approach support vulnerability incident remediation in a PSECO?"* Given this goal and RQ, we selected a research method that enables an in-depth exploration of both the technological and social dimensions within the organizational context. Thus, a participative case study was chosen as research method to help the comprehension of situations investigated, enabling the emergence of new relationships Trinkenreich et al. (2019). In addition, one of the researchers was actively embedded within the organization that owns the PSECO under study, participating directly in the observed processes Baskerville (1997).

In order to use a type of case study as a research strategy, we take into account two foundations based on Yin Yin (2005): i) the investigation of a contemporary phenomenon within its real-life context (the use of GenAI for vulnerability fix in PSECO); and ii) the researcher's access to an event or phenomenon hitherto inaccessible to scientific research.

In close collaboration with practitioners, this participative case study allowed us to validate and refine the PSECO-SafePatch approach, ensuring it was not only technically robust but also aligned with the operational realities of a PSECO. We automated the remediation of vulnerability incidents in a way that directly addressed the recurring challenges faced by the IT management team of a large global organization. To support answering the main RQ, we formulated the following SQ:

SQ1: *"How efficient and effective is our GenAI-based approach for vulnerability remediation in a PSECO?"*. **Rationale**: seeks to understand the extent to which the proposed GenAI-based approach improves efficiency and effectiveness in vulnerability remediation processes within PSECO Cotroneo et al. (2024), Outão et al. (2025), Chatzigeorgiou et al. (2024).

SQ2: *"How do IT managers and developers perceive the PSECO-SafePatch tool as useful and easy of use for vulnerability fix process in the PSECO?"*. **Rationale**: explores the perceptions of IT managers and developers regarding the usefulness and ease of use of the PSECO-SafePatch tool in the vulnerability remediation process within PSECO, inspired by the Technology Acceptance Model (TAM) Davis (1993). TAM is a well-established information systems model that explains how users accept and use a particular technology Davis (1989).

Therefore, SQ can help break down broad questions into smaller parts, enabling a more focused analysis of specific aspects of the phenomenon under study. In our context, the SQ was designed to evaluate the efficiency, the usefulness, and the ease of use of the proposed approach in the organizational context. We defined the following metrics for each SQ, along with their respective operationalization details, as presented below:

- **SQ1. Metric 1. Mean Time per Fix (MTF)**: Represents the average time (in minutes) required to correct a single vulnerability, including review, code modification, validation, and deployment.

$$\text{MTF} = \frac{\text{Total Effort (in hours)} \times 60}{\text{Number of Vulnerabilities Fixed}}$$

- **SQ1. Metric 2. Patch Success Rate (PSR)**: Represents the percentage of AI-generated patches that were accepted and deployed without requiring rework or causing regressions.

$$\text{PSR (\%)} = \frac{\text{Approved Patches without Rework}}{\text{Total Patches Generated}} \times 100$$

- **SQ1. Metric 3. Productivity Gain (PG)**: Represents the percentage of productivity gain obtained by reducing the average time to fix vulnerabilities when using the automated approach compared to the manual method.

$$\text{PG (\%)} = \frac{\text{MTF}_{\text{manual}} - \text{MTF}_{\text{automated}}}{\text{MTF}_{\text{manual}}} \times 100$$

- **SQ2. Metric 4. Perceived Ease of Use (PEoU)**: Represents the median of participants' responses to the PEoU questions, considering that each question was rated using a Likert scale.

$$\text{PEoU} = \text{Median}\left(\left\{P_j \mid j \in \text{Ease of Use Questions}\right\}\right)$$

- **SQ2. Metric 5. Perceived Usefulness (PU)**: Represents the median of participants' responses to the PU questions, considering that each question was rated using a Likert scale.

$$\text{PU} = \text{Median}\left(\left\{P_j \mid j \in \text{Usefulness Questions}\right\}\right)$$

Below, we describe Table 4 that summarizes the metrics used in our evaluation, including their related SQ and the rationale behind each one.

7.4. Planning

We planned the collection of objective perfomance data to address SQ1 focused on indicators such as time, effort, and patch success. The planning step defined the techniques used for data collection and analysis throughout the case study. In line with the scope of our GenAI-based approach for vulnerability remediation, we first identified the need to map the current " as is" CI/CD pipeline, focusing on how security vulnerabilities were detected, prioritized, and fixed within the development process. This mapping provided the baseline necessary to understand operational practices and identify areas for improvement. Details of the existing software development process are available at https://doi.org/10.5281/zenodo.15386005.

We established a structured data collection protocol encompassing vulnerability remediation efforts. We gathered data on: (i) the number of vulnerabilities fixed manually and automatically; (ii) the total effort dedicated to each correction method; and (iii) the resulting mean time per fix. These data points enabled the subsequent calculation of productivity gains and effectiveness metrics associated with the proposed approach.

All data were collected using the organization's internal DevOps tracking tools, based on Jira Software[6] integrated with Bitbucket[7] and Bamboo[8], allowing the management of work items, vulnerability corrections, and build/release activities throughout the software development life-cycle.

Following the collection of objective performance data, we applied a complementary evaluation method to address SQ2, which investigates the perceived usefulness and ease of use of the proposed approach. To that end, we designed a survey-based data collection instrument aimed

[6] https://www.atlassian.com/software/jira
[7] https://www.atlassian.com/software/bitbucket
[8] https://www.atlassian.com/software/bamboo

Table 4
Overview of the evaluation metrics used in the study.

ID	Metric	SQ	Rationale
M1	MTF	SQ1	Measures the average time developers took to fix vulnerabilities, indicating efficiency.
M2	PSR	SQ1	Indicates the percentage of successfully fixed vulnerabilities, showing effectiveness.
M3	PG	SQ1	Compares the time taken in both rounds to show productivity improvement with the tool.
M4	PEoU	SQ2	Captures how easy participants felt it was to use the tool.
M5	PU	SQ2	Captures how useful participants found the tool for fixing vulnerabilities.

at capturing participants' perceptions regarding GenAI-assisted vulnerability remediation within the PSECO context. This feedback complements the quantitative results and provides additional insight into the practical acceptance and usability of PSECO-SafePatch.

The survey consists of an electronic questionnaire to be filled out in 10–15 minutes. It was sent to the participants' e-mails; in this case, experts of the development teams of a large global organization in the context of PSECO. It was divided into 4 blocks. In the first, an introductory text was presented, bringing the objectives of the questionnaire for academic purposes. In the second, the participant should read and agree or disagree with the Informed Consent Term (ICT) before having access to the questions. ICT disclaimer can be found in our supplementary materials at https://doi.org/10.5281/zenodo.15386005.

The third block aimed to characterize the professional profile of the participants. We asked the participants to provide their **Email addresses** if they wished to receive a report of the survey in the future. To gain insights into their educational backgrounds, we inquired about their **academic qualifications**, offering options ranging from high school education to doctoral degrees. Furthermore, participants were requested to specify their **current job title or hierarchical level** within their organization, with options including various levels of software development roles.

To assess the depth of their **professional experience**, we asked participants to indicate the number of years they have been involved in software development or related fields, with categories spanning from 1 to over 20 years of experience. This data was essential to understanding the diverse professional landscapes and experience levels represented in our participant group, allowing us to correlate these factors with other responses throughout the survey.

Finally, the fourth block contained four questions referring to the evaluation of perceived ease of use, and four others related to perceived usefulness, according to the TAM dimensions Davis (1989). Participants answer each question with a value in an ordinal scale: Strongly Agree, Partially Agree, Neutral (Neither Agree nor Disagree), Partially Disagree and Strongly Disagree.

Once each question has been computed, an interpretation regarding the ease of use and usefulness of the tool can be obtained. Due to the high number of possible combinations for the configuration of each response, we chose to separate the interpretation of the ease of use and usefulness. The participants answers spreadsheet will be available in our supplementary materials at https://doi.org/10.5281/zenodo.15386005.

For the refinement of the instruments in this feedback evaluation, a pilot was carried out with two participants. Finally, the form was sent to the potential participants. There was also one last question which the participant could write general comments on the topics covered in the feedback. We defined the questions aligned with the goal of the evaluation, as described in Table 5.

7.5. Execution

From July to August 2024, we applied the PSECO-SafePatch approach in the target organization's CI/CD environment. Throughout its application, we collected performance data from Fortify Static Code Analyzer vulnerability reports, supporting SQ1. During this period, we conducted iterative refinement cycles to ensure accurate data acquisition. Based on preliminary findings, we adjusted the data collection procedures to address inconsistencies.

In parallel, one of the researchers, embedded in the development team, directly observed the remediation workflows using the GenAI-based tool. The researcher accompanied developers during the fixing activities, providing clarification on the tool when needed, and capturing relevant contextual observations about usability, workflow interruptions, and validation steps. These observations provided input for improving future versions of the tool, helping to align it more closely with user needs and operational context.

In addition, the DevOps tracking tools used by the organization provided the official time logs related to vulnerability remediation tasks. These logs were consolidated into structured spreadsheets. This dataset enabled the computation of performance metrics, such as MTF, PSR, and PG. Following the two-month performance data collection period, we also gathered complementary perception-based data, such as PEoU and PU, to address SQ2. To this end, a feedback evaluation form was sent by Email to 18 participants in the organization's development team (developers and technical leaders) who were directly involved in the vulnerability remediation process. In total, 15 responses were submitted, resulting in a response rate of 83.3 %, relative to the targeted development team.

The developers who participated in this feedback evaluation had a solid background as software developers with a focus on areas such as deployment, configuration, and monitoring, as described in Table 6. These developers also held skills for the change management, emergency response, and capacity management of services. In essence, the developers demonstrated a more comprehensive understanding of the life-cycle of the software development process, including the ability to maintain stable operations while implementing necessary fixes in a timely manner. This expertise was important in evaluating the tool's impact on their workflow, particularly in how PSECO-SafePatch aids in streamlining vulnerability fix without disrupting regular development activities.

7.6. Results

This section presents the results of a two-month participative case study conducted in the organization "X", comparing manual vulnerability remediation with the PSECO-SafePatch approach. The goal was to assess the approach's impact on both technical performance and user perception. Thus, we organized the results according to the two SQ. For SQ1, we evaluated efficiency and effectiveness through three performance-related metrics: MTF, PSR, and PG. For SQ2, we evaluated user acceptance through PEoU and PU.

7.6.1. SQ1: How efficient is our genAI-based approach for vulnerability remediation in a PSECO?

The comparison was conducted using two equal two-month periods. The **same development team** was responsible for both phases. Manual corrections were performed first, as part of the team's standard routine, followed by automated corrections during the formal evaluation phase of the PSECO-SafePatch approach. In both cases, vulnerabilities were identified by the same Fortify scanning pipeline and originated from the same applications portfolio. This setup helps ensure that differences

Table 5
Questions derived from **T**AM model (**QT**) to evaluate our tool.

QT#	Description	Dimensions
QT1	I easily learned how to use the tool.	Ease of use
QT2	I used the tool in the way I wanted to.	
QT3	I understood what happened in the interaction with the tool.	
QT4	I easily fixed the vulnerabilities with the tool support.	
QT5	The tool was useful for fixing vulnerabilities.	Usefulness
QT6	The tool allowed me to realize how the use of GenAI can be used in fixing vulnerabilities.	
QT7	The tool improved my performance when fixing vulnerabilities.	
QT8	The tool supported the software development process for fixing vulnerabilities.	

Table 6
Participants profile of development team in the organization "X". Due to privacy reasons, participants were anonymized and referred to "(Px)" where "P" is the Participant and "x" is the ID.

Profile	Skills	ID
Junior developer	Learns basic programming skills and works under supervision. Handles simple tasks and focuses on improving abilities.	P3, P7, P15
Mid-level developer	Works independently on moderately complex problems. Contributes to technical decisions and mentors junior developers.	P2, P6, P8, P11, P14
Senior developer	Leads complex projects, makes critical technical decisions, and ensures system quality. Mentors less experienced developers.	P1, P5, P9, P13
Tech Lead	Oversees the technical direction of the team, aligns technical decisions with business goals, and manages complex project execution.	P4, P10
Principal Engineer	Leads innovation and high-level architectural decisions at an enterprise scale. Mentors technical leaders and influences strategic decisions.	P12

in team composition, workload, and technical environment did not influence the results.

Table 7 summarizes the core results comparing the manual remediation process and the PSECO-SafePatch automated approach. The comparative analysis between manual correction and automated correction by PSECO-SafePatch shows a substantial difference in both time and throughput. The manual method consumed 641 hours to correct 2259 vulnerabilities, while the use of the tool reduced the time to 98 hours, correcting 2288 vulnerabilities. Although the vulnerability sets are not identical, they are comparable in complexity and origin. These findings emphasize the efficiency gains achieved with the automated approach, both in terms of time saved and volume of completed fixes.

The quantitative results presented above reveal consistent improvements in both efficiency and patch quality when using the PSECO-SafePatch tool. Below, we summarize the main findings, highlighting how the adoption of the tool influenced the vulnerability remediation process in practice.

Efficiency Improvement: The **MTF** decreased from **17.03 minutes** (manual) to **2.57 minutes** (automated), reflecting a significant reduction in remediation time. This result indicates that the tool successfully simplified key steps of the fix process, such as patch suggestion and review preparation.

Productivity Interpretation: The **84 %** of **PG** reflects the relative reduction in the average fix time, not a general acceleration of the developer performance. It should be interpreted as a localized efficiency increase within vulnerability remediation tasks, an area where automation using GenAI reduced the need for manual analysis and minimized context switching.

Effectiveness in Patch Quality: Among the 2288 patches generated using PSECO-SafePatch, we achieved a **PSR** of **89 %**, meaning that the vast majority of patches did not require further modification and were successfully deployed. It indicates high reliability of AI-assisted patch suggestions.

These results demonstrate that integrating GenAI into the remediation workflow can significantly reduce the time to respond to vulnerabilities while maintaining high patch quality. The results validate the design goals of PSECO-SafePatch and suggest potential benefits for other PSECO facing similar maintenance challenges.

Table 7
Summary of Evaluation Metrics.

Metric	Manual Process	PSECO-SafePatch Tool (Automated)
Total vulnerabilities fixed	2259	2288
Total effort (hours)	641	98
Mean time per fix (minutes)	17.03	2.57
Productivity gain (%)	—	84 %
Patch success rate (no rework)	Not measured	89 %

Table 8
Educational background of 15 respondents.

Education Level	Qty	Percentage (%)
Bachelor's Degree	8	54 %
Postgraduate Certificate	5	33 %
Master's Degree	2	13 %
High School	0	0 %
Doctorate	0	0 %

7.6.2. SQ2: How do IT managers and developers perceive the PSECO-Safepatch tool as useful and easy of use for vulnerability fix process in the PSECO?

Feedback and communication of results are integral parts of the evaluation cycle Lo et al. (2015). We evaluated the perceptions of IT managers and developers about the PSECO-SafePatch tool, focusing specifically on its perceived ease of use and usefulness in supporting vulnerability remediation tasks.

This assessment is grounded in the TAM model, which offers a structured understanding of how users evaluate the usability and practical value of the system in real world scenarios Davis (1989). The demographic data below was collected to validate the relevance and credibility of the participant sample, ensuring that the feedback used in the evaluation came from practitioners with sufficient experience and contextual familiarity.

The educational background of the respondents is presented in Table 8, showing that most participants hold undergraduate or higher-level degrees. None reported having only a *High School* education or a *Doctorate*, which indicates a generally high level of education among participants.

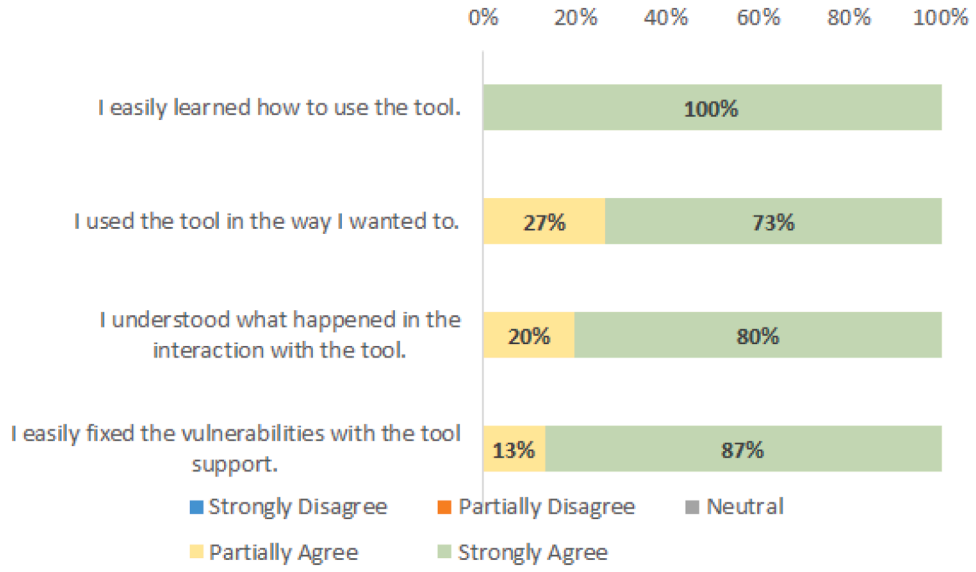

Fig. 10. PSECO-SafePatch tool regarding the ease of use tasks.

Table 9
Hierarchical level of 15 respondents.

Position	Qty	Percentage (%)
Mid-level Developer	5	33 %
Senior Developer	4	27 %
Junior Developer	3	20 %
Tech Lead	2	13 %
Principal Engineer	1	7 %

Table 10
Professional experience of 15 respondents.

Years of Experience	Qty	Percentage (%)
1 to 5 years	3	20 %
5 to 10 years	5	33 %
10 to 15 years	4	27 %
15 to 20 years	2	13 %
More than 20 years	1	7 %

Table 9 shows the hierarchical distribution of the respondents within the organization. These results indicate a diverse range of experience and seniority within the organization, with a strong representation of mid-level and senior developers, alongside a smaller proportion of leadership roles.

Table 10 summarizes the professional experience of the respondents, showing a concentration of participants with 5 to 15 years of experience in software development or related areas. These results indicate a diverse range of experience levels, with a strong representation of mid-career professionals.

Regarding the tool's ease of use, the results showed an overall highly positive perception, as illustrated in Fig. 10. Participants unanimously found the tool intuitive and easy to adopt (**QT1**), and most were able to use it as intended (**QT2**). The majority also understood the tool's feedback clearly (**QT3**), although a few respondents indicated that additional clarity could be beneficial. Finally, the tool was widely perceived as effective in assisting with vulnerability remediation (**QT4**), with minor difficulties reported by a small subset of users.

Regarding the tool's usefulness, the feedback was also positive, as illustrated in Fig. 11. Participants unanimously agreed that the tool was useful in supporting vulnerability remediation tasks (**QT5**) and in improving their performance during the fixing process (**QT7**). Additionally, respondents recognized that the tool helped them realize the potential of GenAI for vulnerability management (**QT6**). Most participants also affirmed that the tool supported broader software development activities (**QT8**), although a few indicated opportunities for further enhancement in this area.

Finally, participants pointed out comments on the tool as well as considerations for improvements. Several participants praised PSECO-SafePatch tool for reducing the time spent on vulnerability fix. As strengths, P4 mentioned: *"The tool's ability to automatically identify and prioritize vulnerabilities saved us countless hours, allowing the team to focus on new development without worrying about security gaps"*. P1 highlighted the intuitive interface and clear feedback provided during the remediation process: *"It was easy to understand what the tool was doing and how it was fixing the issues"*. P10 reinforced the positive perception by acknowledging the contribution of GenAI, commenting: *"I was impressed with how the tool demonstrated the potential of genAI in fixing vulnerabilities"*.

However, participants also suggested areas for improvement. P7 recommended evolving the tool to integrate as a quality gate within the CI/CD pipeline, allowing it to block deployments in the presence of critical vulnerabilities: *"The tool must evolve to integrate as a quality gate in CI/CD pipeline, allowing it to block deployments if critical vulnerabilities were detected"*. P2 suggested extending the integration capabilities, highlighting the benefit of offering the tool as a plugin directly within developers' IDE: *"While the tool was generally easy to use, I expected it to offer more integration options, such as being available as a plugin directly within the developer's IDE"*. P12 further recommended enhancements to support heterogeneous environments, particularly microservice-based architectures: *"Based on my experience, I would recommend customization options for more complex environments, especially when dealing with microservice architectures. In microservices environments, it is common for each service to be implemented using different programming languages"*.

To enrich the qualitative feedback, we highlight that, contrary to concerns about junior developers fearing job displacement by AI tools Le (2025), participants in this role (P3, P7, and P15) expressed high acceptance and perceived usefulness of the proposed solution. This underscores the importance of transparency in fostering trust. P15 emphasized the tool's intuitiveness and its positive impact on productivity, stating: *"The tool exceeded my expectations. I was able to use the tool intuitively, and it really sped up the vulnerability fixing process"*.

As a complement to the qualitative feedback, we computed measures of central tendency for the quantitative questions related to ease

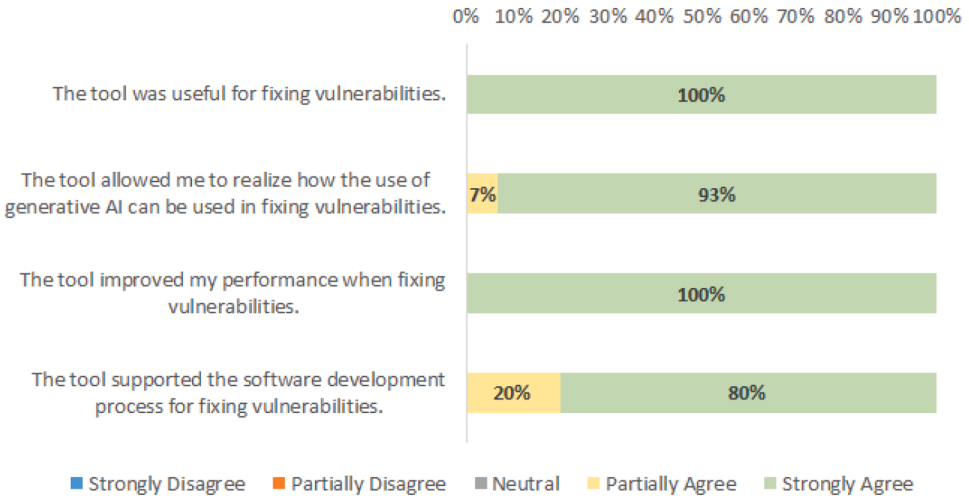

Fig. 11. PSECO-SafePatch tool regarding the usefullness of tasks.

of use and usefulness. Given that responses were collected using a Likert scale, which produces ordinal rather than interval data, the median was chosen as the most appropriate measure of central tendency, instead of the mean Wu and Leung (2017). Each Likert response was mapped to a numerical value from 1 (Strongly Disagree) to 5 (Strongly Agree).

Initially, the median was calculated individually for each question within the ease of use and usefulness dimensions. Next, the overall median for each dimension was determined based on the medians of their respective questions. For both dimensions, **easy of use (PEoU)** and **useful (PU)**, the final median value was 5, indicating that participants strongly agreed with the tool's ease of use and usefulness. Considering these results, the findings point to a highly positive perception across both evaluated dimensions, with consistent user satisfaction regarding the tool's intuitiveness and its practical value in supporting vulnerability remediation tasks.

8. Discussion

To provide in-depth discussion, we triangulated the results of our case study on the development of the PSECO-SafePatch approach to automate vulnerability fixes in the CI/CD pipeline and the existing literature on other approaches. This section is organized into subsections that highlight the main findings, discuss their implications for practice and research, and position our contributions in light of related work to support a more transparent and structured interpretation of the results.

8.1. Main findings

Our work revealed the following findings regarding the application of a GenAI-based approach for vulnerability remediation within the PSECO environment. Our findings are structured according to the features and capabilities identified in the comparative analysis of GenAI-based vulnerability remediation approaches (see Table 1), highlighting how PSECO-SafePatch addresses challenges observed in the literature.

Workflow Integration Compatibility. Unlike approaches such as *GenProg* Le Goues et al. (2011) and *RepairAgent* Bouzenia et al. (2024), which operate in isolation and lack direct integration with enterprise pipelines, and GitHub Copilot, which is primarily limited to IDE-level support, PSECO-SafePatch was designed to natively integrate into the CI/CD pipeline of a PSECO. By connecting with Fortify's static code analysis and DevOps tracking tools, it enabled vulnerabilities to be patched, validated, and deployed within the organization's existing CI/CD pipeline.

This compatibility may be critical in PSECO environments, where software development is governed by strict compliance, traceability, and change management protocols. PSECO often involves interdependent applications, domain-specific governance structures, and audit requirements that limit the use of generic tools Outão et al. (2025). Therefore, embedding the remediation process into the CI/CD workflow not only ensures scalability but also aligns with internal policies and enterprise controls, enhancing trust and adoption.

This finding is consistent with insights from the study by Monperrus Monperrus (2018), which identifies the integration with industrial toolchains as one of the primary obstacles to the real-world adoption of automated repair solutions. Some academic approaches remain disconnected from the complex environments in which developers operate. In contrast, PSECO-SafePatch addresses this challenge by enabling automated patching directly within enterprise workflows.

Security and Privacy Constraints. Most existing GenAI-assisted development tools, such as *Github Copilot* Dakhel et al. (2023) and *RepairAgent* Bouzenia et al. (2024), rely on public APIs or cloud-hosted services, which raises concerns in enterprise environments regarding source code confidentiality. These concerns are amplified in PSECO, where regulatory compliance and strict data governance are fundamental Outão et al. (2025). On the other hand, PSECO-SafePatch is deployed within a private Azure OpenAI instance, ensuring that no confidential data leaves the organizational perimeter. This privacy-preserving setup directly addresses one of the key barriers to GenAI adoption in regulated environments such as PSECO.

This finding aligns with concerns raised in the study of Banh et al. (2025), which highlights two relevant factors: (i) the difficulty in adapting copilots to specific organizational contexts; and (ii) organizational barriers to widespread technology adoption. By running entirely within a secure and isolated environment, PSECO-SafePatch mitigates these risks.

Complementary Tool (human-in-the-loop). PSECO-SafePatch incorporates a human-in-the-loop approach by requiring human validation before deploying AI-generated patches. This contrasts with fully autonomous systems such as *GenProg* Le Goues et al. (2011) and *RepairAgent* Bouzenia et al. (2024), which may apply fixes without structured oversight. Although *Github Copilot* Dakhel et al. (2023) allows user interaction, its passive suggestions lack an enforced verification step.

Our findings highlight that combining automation with human verification enhances trust and accountability in sensitive environments. This aligns with the principles of human-in-the-loop assertion generation discussed by Zamprogno et al. (2022). The authors demonstrated that involving humans in the decision loop improves the correctness

and relevance of automatically generated software assertions. Furthermore, we observed that hallucination, where GenAI fabricates incorrect code suggestions, remains a key challenge Choudhuri et al. (2024). By mandating human oversight, PSECO-SafePatch mitigates these risks and ensures that critical security patches meet organizational standards before integration.

Task Complexity. While *GenProg* Le Goues et al. (2011) focuses on isolated defects and *RepairAgent* Bouzenia et al. (2024) operates within prompt-based scenarios, PSECO-SafePatch was designed to handle the architectural complexity typical of PSECO Outão et al. (2025). In such environments, multiple interdependent software assets, often spanning several services and versions, require coordinated and context-aware remediation. This is a critical requirement in PSECO, where fixes to one module can affect others due to shared dependencies Costa et al. (2024b).

A recent study Lyu et al. (2024) highlighted key limitations in current GenAI-based repair methods, including the difficulty of generating safe code, the focus on simple defects, and the dependence on weak test suites for validation, which can lead to overfitting. By integrating remediation into a broader architectural view and involving developers in the loop, PSECO-SafePatch offers a more reliable solution to these challenges.

Code Improvement and Maintenance. Concerns have been raised in the literature regarding the impact of GenAI tools on code quality and long-term maintainability. Developers often report uncertainty about the correctness and clarity of AI-generated code, which may hinder understanding and gradually introduce subtle technical debt Banh et al. (2025).

Our study suggests that PSECO-SafePatch effectively addresses these concerns. With a PSR of 89 %, the tool consistently produced fixes that adhered to internal code quality standards, reinforcing their maintainability and reliability. Rather than focusing solely on productivity, as seen in *Github Copilot* Dakhel et al. (2023) or test-passing criteria, as emphasized in *GenProg* Le Goues et al. (2011), our approach supports the gap between automation and sustainable SE.

Efficiency and Automation. Recent studies have shown that GenAI tools are reshaping the landscape of software development, influencing how engineers write, test, debug, and maintain code Banh et al. (2025), Carleton et al. (2024). In particular, developers increasingly rely on copilots for tasks such as writing boilerplate code, generating tests, refactoring, and retrieving documentation. These behaviors suggest a shift toward AI-supported practices that prioritize developer productivity.

PSECO-SafePatch achieved a notable PG of 84 % by reducing the MTF from 641 to 98 hours. Unlike *GenProg* Le Goues et al. (2011), which depends on comprehensive test suites to generate patches and *GitHub Copilot* Dakhel et al. (2023), which focuses on code assistance without automation of the CI/CD remediation flow, PSECO-SafePatch offers an integrated pipeline for identifying, prioritizing, and patching vulnerabilities at scale. This capability bridges the gap between productivity gains and compliance requirements, demonstrating the feasibility of GenAI-based remediation in PSECO.

Trust and Reliability. Our approach incorporates validation steps, auditability, and rollback capabilities, allowing organizations to trace each AI-generated fix. This level of traceability is absent in the other approaches analyzed. By enforcing structured human validation and leveraging Fortify integration for compliance verification, PSECO-SafePatch strengthens organizational trust in AI-generated code changes.

To address concerns of trust and reliability, our approach adopts a human-in-the-loop strategy that places experienced developers in a supervisory role throughout the patching process Zamprogno et al. (2022). Rather than positioning GenAI as an autonomous actor as seen in approaches such as *GenProg* Le Goues et al. (2011) and *RepairAgent* Bouzenia et al. (2024), our approach promotes collaboration, enabling developers to critically assess, adjust, and approve GenAI-generated fixes before integration. It mitigates two major risks highlighted by Banh et al. (2025): i) cognitive overload; and ii) overreliance. The first arises when developers are overwhelmed by the constant need to verify the accuracy of GenAI suggestions, which can lead to fatigue and reduced effectiveness. The second refers to the erosion of essential engineering skills and critical thinking when teams blindly trust GenAI outputs. By supporting decision-making rather than replacing it, PSECO-SafePatch improves code reliability and preserves the developers' expertise, ensuring that trust is built on both technological rigor and human judgment.

Another relevant point that emerged from our findings relates to how junior developers perceived the tool. Although industry narratives sometimes suggest that less experienced professionals may resist AI-based automation due to fear of role displacement Le (2025), our study revealed the opposite. Participants in junior roles exhibited high levels of trust in the tool, describing it as intuitive, helpful, and even exceeding expectations in some cases. This suggests that when AI tools are placed transparently as aids rather than replacements, they may be welcomed even by those whose tasks are more susceptible to automation.

8.2. Implications for practitioners

The results of our work highlight several implications for software engineers, software architects, developers, and IT managers operating in PSECO. While GenAI-based automation proved effective in improving productivity, its use raises concerns that must be carefully managed in practice.

- **Productivity vs. Knowledge Retention.** The adoption of GenAI-based remediation tools can improve productivity. However, excessive reliance can erode developer technical expertise and reduce business domain knowledge, both of which are critical in complex PSECO environments. Organizations should adopt practices that balance automation with learning continuity.
- **Avoiding Developer Passivity.** Continuous exposure to AI suggestions can make developers overconfident or passive. Enforcing human-in-the-loop validation reinforces critical thinking, preserves team accountability, and sustains managerial trust in human decision-making.
- **Beyond Benchmarks.** Several repair tools succeed on short datasets but fail in enterprise environments. Our findings points out the importance of designing and evaluating GenAI tools within real-world contexts that involve legacy systems, security policies, and interdependent components.
- **Code Robustness and Safety.** Automated remediation must ensure that fixes are secure, correct, and do not reintroduce errors. Our findings suggests rollback mechanisms and validation protocols to ensure robustness, going beyond the behavior of assistive autocomplete tools.

While not a formal result, we observed that separating remediation work into focused squads mirrored principles from "two-pizza team" models Denning (2019).

In this organization, a squad refers to a small, cross-functional team, typically aligned with agile practices, responsible for the development and maintenance of a specific set of applications or services Ly et al. (2025). This agile team structure, centered on specialization, autonomy, and ownership, may further support the integration of GenAI tools into organizations. By isolating vulnerability remediation within dedicated squads, product teams can maintain their delivery flow without being interrupted by urgent security demands. This separation of concerns promotes operational efficiency while preserving focus on business priorities.

Take-home message for practitioners.

Adopting GenAI-based tools for vulnerability remediation in enterprise environments should aim not to replace developers, but to em-

power them. Our findings reinforce that such tools must be carefully integrated into real-world workflows through human-in-the-loop validation, traceability mechanisms, and contextualized feedback. This balance enhances trust and adoption, while also preventing the erosion of developers' technical skills by keeping them engaged in decision-making processes and enabling gains in productivity and software security.

8.3. Implications for researchers

Our findings also offer valuable insights for the research community. As GenAI gains momentum in software engineering, it is critical to go beyond technical benchmarks and understand how these tools are adopted and evaluated in real-world scenarios.

- **Sociotechnical Framing.** Our work reinforces the importance of viewing GenAI interventions as sociotechnical systems. Beyond technical performance, future work should explore how GenAI tools reshape workflows, team dynamics, and trust in enterprise environments, especially in complex and regulated domains such as PSECO.
- **Addressing the Evidence Gap.** Despite their popularity, few empirical studies have examined the deployment of GenAI tools in regulated domains such as PSECO. Our work helps explore this gap, but larger and more diverse cases are needed to understand the broader implications of GenAI adoption.

Take-home message for researchers.

Most prior studies focus on technical accuracy or performance benchmarks of GenAI tools. However, real-world adoption in enterprise environments depends on human trust, organizational workflows, and contextual constraints. Our findings suggest that researchers should investigate not only what GenAI tools generate, but also how they are understood, validated, and integrated by practitioners. Future studies should explore these sociotechnical dynamics across diverse SECO.

8.4. Current usage and informal developments

Although the formal case study finished in August 2024, we found it important to briefly report on the observed developments that followed the evaluation phase. However, we should clarify that this follow-up usage was not systematically investigated within the methodological boundaries of the participative case study Yin (2005).

We collected initial insights through a follow-up meeting held in April 2025, involving participants from the original evaluation phase. The meeting was organized to gather impressions about the continued use of the PSECO-SafePatch tool. Our intention was not to perform a new research design, but rather to informally observe whether the PSECO-SafePatch approach had organically integrated into existing workflows.

The meeting included nine (P1, P2, P3, P4, P7, P9, P11, P12, and P14) of the 15 participants in the evaluation phase, ensuring that at least one practitioner from each predefined profile (junior developer, mid-level developer, senior developer, tech lead, and principal engineer) was present. During the discussion, participants shared their experiences with the tool in the eight months following the evaluation in terms of usability, integration, and perceived value. Below are some illustrative quotes from participants:

- P3: *"The suggestions of the tool are still part of my routine. They help me get started faster, especially during security cycles"*. This sentence highlights the impact of the tool on daily activities, especially by accelerating initial remediation work during recurring audit routines;
- P14: *"The tool became part of our CI/CD for certain systems. It sends us the suggestions and manual intervention was significantly reduced"*. This statement emphasizes the tool's successful integration into the CI/CD pipeline, with benefits in terms of automation and efficiency;
- P9: *"We use the tool mainly in recurring vulnerability scenarios. It does not solve everything, but it is a shortcut when the team is under pressure"*. This reflection illustrates the tool's perceived value in high-pressure situations, where speed and guidance are especially critical;
- P4: *"The tool worked well because it only required minor adjustments in our vulnerability remediation process"*. This opinion underscores the low barrier to adoption, suggesting that the tool fit naturally within existing workflows with minimal disruption; and
- P12: *"We were skeptical at first, but now we trust it to generate baseline patches. Our developers refine them, and that's good enough for our current needs"*. This perception highlights the development of trust over time, with the tool now seen as a reliable starting point for human-in-the-loop refinement.

While this current usage report was not formally planned as part of the original participative case study Yin (2005), the observed continuity of use highlights the practical viability and alignment of the tool with existing processes. Together, the experiences gathered during the participative case study and the insights from the current usage may establish the base for a new phase of research.

Thus, this phase can create opportunities for future research using the action research approach. Action research is a collaborative method in which the researchers and practitioners co-design interventions through iterative cycles of planning, acting, observing, and reflecting Baskerville and Wood-Harper (1996). This research method is especially appropriate for dynamic and complex organizational settings, where both improvements and knowledge development can occur simultaneously. Moreover, the transition also aligns with technology transfer strategies in practice, where sustained use of a solution in real environments fosters long-term collaboration between academia and industry Gorschek et al. (2006).

Given the sustained use of the tool, the active role of practitioners, and the close involvement of the researcher, a follow-up study based on action research would allow us to systematize the evolution of practices and better understand the long-term implications of applying GenAI for vulnerability incident remediation in critical development environments, such as PSECO.

8.5. Contributions

Our work provides multiple contributions to the advancement of research and practice in PSECO and GenAI-enabled SE:

- **Practical Contribution.** We present PSECO-SafePatch, an approach composed of two complementary elements: (i) a structured vulnerability remediation process designed to integrate GenAI capabilities into the existing CI/CD pipeline of a PSECO; and (ii) a web-based tool developed to instantiate activities of this process. Through a participative case study in a large global organization, we demonstrate its ability to automate vulnerability remediation while preserving delivery cadence.
- **Research Contribution.** The proposed architecture combines static analysis tools (Fortify), GenAI services (OpenAI), structured human validation, and rollback capabilities. This integration supports secure, traceable, and policy-compliant remediation processes tailored to regulated environments. Beyond its technical implementation, this architecture reflects a research contribution, as it was shaped through successive cycles of design, application, and evaluation within the daily practices of a real organizational environment.

  Furthermore, we organized the artifacts associated with this contribution, which include architecture diagrams, requirements, and code samples into a structured "replication package" available as supplementary material at https://doi.org/10.5281/zenodo.15386005. This package supports transparency, fosters reuse, and facilitates replication of our approach in future research.
- **Scientific Contribution.** We apply a research strategy that combines the GQM framework with a participative case study. This approach offers a replicable blueprint for evaluating GenAI tools in complex and regulated domains, such as PSECO. More than a

methodological choice, it contributes scientifically by demonstrating how empirical rigor can be maintained even in enterprise environments.

- **Sociotechnical Contribution.** We highlight the risk of knowledge erosion, both technical and domain-specific, when GenAI tools are used without human oversight. By incorporating a human-in-the-loop validation model, our approach reinforces developers' critical thinking, maintains their role in decision-making, and preserves organizational trust. This reflection calls attention to the need for responsible integration of GenAI in enterprise workflows, especially in contexts where overreliance can compromise long-term system sustainability and team autonomy.

Together, these contributions support both practitioners and researchers in navigating the adoption of GenAI in PSECO context. For practitioners, the work provides a concrete approach for automating vulnerability remediation while preserving compliance, developer autonomy, and delivery cadence. For researchers, it offers methodological guidance and sociotechnical insights that can inform future studies on the integration, governance, and human impact of GenAI in complex and regulated software environments.

9. Threats to validity and credibility

The reliability of the results is directly linked to the validity of the study. Every study has threats that should be addressed and considered together with the results, considering the classification proposed in Runeson et al. (2012). In the following, we will discuss potential threats as well as strategies to mitigate them in our work.

Internal credibility addresses whether the findings reflect the reality of the organization's vulnerability correction process. One threat arises from the fact that one of the researchers is an active participant in the observed process. While this insider perspective offers a deep understanding of the organizational context, it could lead to bias on certain findings. To mitigate this, the work incorporated feedback and validation from other stakeholders and practitioners within the organization, ensuring that the results align with the organizational experience. Additionally, the use of a structured GQM approach helps systematically link the research questions, objectives, and metrics, using such a framework for minimizing bias.

Regarding **internal validity**, one limitation concerns the control of confounding variables when comparing manual and automated fixes. To mitigate this, all vulnerabilities used in the evaluation were sourced from the same Fortify scan pipelines, applied to similar applications within the same J2EE technology stack. We selected comparable vulnerability types, such as SQL injection, cross-site scripting (XSS), and hard-coded credentials, based on Fortify severity ratings and their location in the codebase (e.g., controllers, data access objects, utility classes). However, we acknowledge that the actual complexity of a fix may also depend on contextual factors (e.g., architectural dependencies, business logic, or module coupling), which are more difficult to quantify.

Another **internal validity** concern relates to developer skill level and experience. All participants had over five years of experience in SE, worked in the same organizational unit, and were familiar with the systems, tools, and codebase used in the study. These factors may reduce the variability in baseline expertise. However, we acknowledge that skill improvement during the evaluation period may have influenced the results. Rather than treating this as noise, we interpret it as a desirable outcome. The tool provided context-aware suggestions and reusable fix patterns, which facilitated learning and helped standardize secure coding practices. Nonetheless, since no formal tracking of skill evolution was conducted, this remains a limitation in our work.

External validity concerns the extent to which the findings can inform similar contexts. Although this study was conducted within a PSECO, some managerial and technical practices, such as vulnerability management and CI/CD pipelines, are shared between different organizational settings. Thus, our results may serve as useful references for other practitioners facing similar challenges. In line with Flyvbjerg Flyvbjerg (2006), we recognize that case studies can offer practical insights that may be relevant beyond their immediate context, providing examples that inform future research and practice.

As **construct validity**, we relied on the organization's DevOps tracking tools and developer feedback to calculate time spent per fix. However, informal coordination or interruptions may not be fully captured in these logs. In addition, although the definition of "completed fix" was standardized across both periods, some variation may remain in how developers reported effort or marked closure status. These factors may affect the precision of how the construct of productivity was operationalized.

Conclusion validity concerns whether the observed improvements can be reliably attributed to the use of the PSECO-SafePatch tool. Both periods were equivalent in duration (2 months) and involved the same development team working on the same systems, using the same CI/CD pipelines and tools. These controls reduce threats from differences in team experience, tooling, or project scope. Nonetheless, confounding variables, such as the natural improvement of developer skills or parallel organizational changes, may have influenced the results. To mitigate this, we used consistent and objective quantitative metrics (e.g., time per fix and number of vulnerabilities fixed), and ensured that all activities were tracked through internal logs of the DevOps tools.

10. Conclusion and future work

Our work investigates how a GenAI-based approach can support vulnerability remediation in PSECO while preserving delivery cadence in CI/CD pipelines. Using a participative case study as research method, we adopted the GQM guidelines to identify the goals, research questions, and metrics. These guidelines provided a structured and in-depth assessment of the efficiency and effectiveness of GenAI-based vulnerability remediation.

The main outcome of this research was the development of PSECO-SafePatch, an approach designed to address the challenges of vulnerability remediation within a PSECO. This approach consists of two complementary elements: (i) a structured remediation process that integrates GenAI capabilities into the organization's existing CI/CD pipeline; and (ii) a web-based tool developed to instantiate parts of this process by connecting Fortify static code analysis to LLM-based patch generation services. Together, these elements enable the automation of vulnerability fixes in a secure, traceable, and disruptive way.

The results demonstrated improvements in productivity and quality. Compared to the manual baseline, PSECO-SafePatch reduced the average time to fix vulnerabilities from 17.03 minutes to 2.57 minutes, corresponding a 84 % productivity gain, while maintaining a low rate of regressions and additional failures. These outcomes validate the potential of GenAI-based automation to enhance both the speed and reliability of software remediation tasks in enterprise environments.

Our findings also revealed that both developers and IT managers perceived the tool as useful and easy to use, encouraging its adoption. The human-in-the-loop validation mechanisms were positively received, as they helped reduce cognitive overload and preserved engineers' critical thinking. Overall, the combination of consistent quantitative performance and positive user perception reinforces the feasibility and value of integrating GenAI-based automation into regulated and interdependent software ecosystems, such as PSECO.

Although the findings are promising, there are several directions for future work. A relevant path involves conducting an action research study to systematize the long-term use and iterative evolution of the PSECO-SafePatch approach in practice. The study would enable researchers and practitioners to collaboratively refine the solution over time through iterative cycles of planning, implementation, and observation, as recommended in organizational research settings. This approach could also help uncover the patterns of human-in-the-loop collaboration,

integration barriers, and contextual strategies that emerge in real-world usage.

Furthermore, future research may explore the application of the approach in organizational contexts outside the financial sector. It is also worth investigating how GenAI can support the management of technical debt, automate repetitive and cognitively demanding engineering tasks, and enable the use of multimodal models capable of processing code, documentation, and visual artifacts simultaneously. Finally, data-driven engineering, which makes use of historical data and real-time telemetry, holds promise for improving the contextualization and validation of AI-generated patches. This approach opens new opportunities for intelligent decision making in the development of secure software.

Statements and Declarations

- Funding. This work was supported by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001 and Grant 88887.955223/2024-00, CNPq (Grant 316510/2023-8), FAPERJ (Grant E-26/ 204.404/2024), and UNIRIO.
- Competing Interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.
- Author Contributions. Luiz Alexandre M. Costa and Awdren Fontão: Data curation, Formal analysis, Investigation, Methodology, Resources, Writing - Original Draft, Visualization. Rodrigo Pereira dos Santos and Alexander Serebrenik: Conceptualization, Funding acquisition, Project administration, Validation, Writing - Review & Editing.
- Data availability. Data sharing for this article is available at https://doi.org/10.5281/zenodo.15386005.

CRediT authorship contribution statement

Luiz Alexandre Costa: Writing – original draft, Visualization, Resources, Methodology, Investigation, Formal analysis, Data curation; **Awdren Fontão:** Writing – original draft, Visualization, Resources, Methodology, Investigation, Formal analysis, Data curation; **Rodrigo Pereira dos Santos:** Writing – review & editing, Validation, Project administration, Funding acquisition, Conceptualization; **Alexander Serebrenik:** Writing – review & editing, Validation, Supervision, Project administration, Funding acquisition, Conceptualization.

Data availability

Data will be made available on request.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Luiz Alexandre Costa reports financial support was provided by CAPES (Financial Code 001 and Grant 88887.955223/2024-00). Rodrigo Pereira dos Santos reports financial support was provided by CNPq (Grant 316510-2023-8). Rodrigo Pereira dos Santos reports financial support was provided by FAPERJ (Grant E-26-204.404–2024. The other authors declare no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001 and Grant 88887.955223/2024-00, CNPq (Grant 316510/2023-8), FAPERJ (Grant E-26/ 204.404/2024), and UNIRIO.

References

Agutter, C., 2020. ITIL Foundation Essentials ITIL 4 Edition-The Ultimate Revision Guide. IT Governance Publishing Ltd.

Ahmed, I., Aleti, A., Cai, H., Chatzigeorgiou, A., He, P., Hu, X., Pezzè, M., Poshyvanyk, D., Xia, X., 2025. Artificial intelligence for software engineering: the journey so far and the road ahead. ACM Trans. Softw. Eng. Method. .

Alves, C., Oliveira, J., Jansen, S., 2017. Understanding governance mechanisms and health in software ecosystems: a systematic literature review. In: International Conference on Enterprise Information Systems. Springer, pp. 517–542.

Anderson, P., Kot, Ł., Gilmore, N., Vitek, D., 2019. Sarif-enabled tooling to encourage gradual technical debt reduction. In: 2019 IEEE/ACM International Conference on Technical Debt (TechDebt). IEEE, pp. 71–72.

Austin, A., Holmgreen, C., Williams, L., 2013. A comparison of the efficiency and effectiveness of vulnerability discovery techniques. Inf . Softw. Technol. 55 (7), 1279–1288.

Banh, L., Holldack, F., Strobel, G., 2025. Copiloting the future: how generative AI transforms software engineering. Inf. Softw. Technol. , 107751.

Basili, V.R., 1992. Software modeling and measurement: the Goal/Question/Metric paradigm. University of Maryland at College Park.

Baskerville, R.L., 1997. Distinguishing action research from participative case studies. J. Syst.Inf. Technol. 1 (1), 25–45.

Baskerville, R.L., Wood-Harper, A.T., 1996. A critical perspective on action research as a method for information systems research. J. Inf. Technol. 11 (3), 235–246.

Bosch, J., 2009. From software product lines to software ecosystems. In: Systems and Software Product Line Conference. Vol. 9, pp. 111–119.

Bouzenia, I., Devanbu, P., Pradel, M., 2024. Repairagent: An autonomous, LLM-based agent for program repair. arXiv preprint arXiv:2403.17134 .

Carleton, A., Falessi, D., Zhang, H., Xia, X., 2024. Generative AI: redefining the future of software engineering. IEEE Softw. 41 (6), 34–37.

Chatzigeorgiou, A., Ahmed, I., Cai, H., Pezzè, M., Poshyvanyk, D., 2024. Artificial intelligence for software engineering: the journey so far and the road ahead. ACM Trans. Softw. Eng. Method. 34 (9).

Chawdhury, T.K., 2024. Beyond the Falcon: A Generative AI Approach to Robust Endpoint Security. Technical Report. Technical report.

Chen, L., 2017. Continuous delivery: overcoming adoption challenges. J. Syst. Softw. 128, 72–86.

Choudhuri, R., Liu, D., Steinmacher, I., Gerosa, M., Sarma, A., 2024. How far are we? the triumphs and trials of generative ai in learning software engineering. In: Proceedings of the IEEE/ACM 46Th International Conference on Software Engineering, pp. 1–13.

Costa, L.A., Fontão, A., Santos, R., 2021. Toward proprietary software ecosystem governance strategies based on health metrics. IEEE Trans. Eng. Manage. 69 (6), 3589–3603.

Costa, L.A., Fontão, A., Santos, R.P., Serebrenik, A., 2024a. Towards an incident management framework in proprietary software ecosystems. https://arxiv.org/abs/2410.09320.

Costa, L.A., Outão, J.C., Simões, J.E., Santos, R. P.d., 2024b. Towards software asset management in proprietary software ecosystems: a participative case study. In: Proceedings of the XXIII Brazilian Symposium on Software Quality, pp. 69–80.

Cotroneo, D., Foggia, A., Improta, C., Liguori, P., Natella, R., 2024. Automating the correctness assessment of AI-generated code for security contexts. J. Syst. Softw. , 112113.

Dakhel, A.M., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M.C., Jiang, Z. M.J., 2023. Github copilot ai pair programmer: asset or liability? J. Syst. Softw. 203, 111734.

Davis, F.D., 1989. Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Q. , 319–340.

Davis, F.D., 1993. User acceptance of information technology: system characteristics, user perceptions and behavioral impacts. Int. J. Man Mach. Stud. 38 (3), 475–487.

Davis, J., Daniels, R., 2016. Effective DevOps: building a culture of collaboration, affinity, and tooling at scale. " O'Reilly Media, Inc.".

Denning, S., 2019. How amazon practices the three laws of agile management. Strateg. Leadership 47 (5), 36–41.

Dissanayake, N., Zahedi, M., Jayatilaka, A., Babar, M.A., 2022. Why, how and where of delays in software security patch management: an empirical investigation in the healthcare sector. Proc. ACM Human-Comput. Interact. 6 (CSCW2), 1–29.

Doynikova, E., Kotenko, I., 2016. Countermeasure selection based on the attack and service dependency graphs for security incident management. In: Risks and Security of Internet and Systems: 10Th International Conference, CRiSIS 2015, Mytilene, Lesbos Island, Greece, July 20–22, 2015, Revised Selected Papers 10. Springer, pp. 107–124.

Ebert, C., Louridas, P., 2023. Generative AI for software practitioners. IEEE Softw. 40 (4), 30–38.

Fenz, S., Heurix, J., Neubauer, T., Pechstein, F., 2014. Current challenges in information security risk management. Inf. Manag. Comput. Secur. 22 (5), 410–430.

Fitzgerald, B., Stol, K.-J., 2017. Continuous software engineering: a roadmap and agenda. J. Syst. Softw. 123, 176–189.

Flyvbjerg, B., 2006. Five misunderstandings about case-study research. Q. Inq. 12 (2), 219–245.

Gorschek, T., Garre, P., Larsson, S., Wohlin, C., 2006. A model for technology transfer in practice. IEEE Softw. 23 (6), 88–95.

Govindaras, B., Wern, T.S., Kaur, S., Haslin, I.A., Ramasamy, R.K., 2023. Sustainable environment to prevent burnout and attrition in project management. Sustainability 15 (3), 2364.

Hilario, E., Azam, S., Sundaram, J., Imran Mohammed, K., Shanmugam, B., 2024. Generative AI for pentesting: the good, the bad, the ugly. Int. J. Inf. Secur. 23 (3), 2075–2097.

Korzynski, P., Mazurek, G., Krzypkowska, P., Kurasinski, A., 2023. Artificial intelligence prompt engineering as a new digital competence: analysis of generative AI technologies such as chatGPT. Entrepreneur. Bus. Econ. Rev. 11 (3), 25–37.

Le, H., 2025. The deskilling of software development and the impact of AI chatbots on programmers' skills and roles. In: Generative AI in Software Engineering. IGI Global Scientific Publishing, pp. 397–426.

Le Goues, C., Nguyen, T., Forrest, S., Weimer, W., 2011. Genprog: a generic method for automatic software repair. IEEE Trans. Softw. Eng. 38 (1), 54–72.

Lenarduzzi, V., Saarimäki, N., Taibi, D., 2019. The technical debt dataset. In: Proceedings of the Fifteenth International Conference on Predictive Models and Data Analytics in Software Engineering, pp. 2–11.

Leão, R., Ayach, F., Lameirão, V., Fontão, A., 2024. A prompt engineering-based process to build proto-personas during lean inception. In: Anais Do XXXVIII SimpóSio Brasileiro De Engenharia De Software. SBC, Porto Alegre, RS, Brasil, pp. 585–591. https://doi.org/10.5753/sbes.2024.3562

Lo, D., Nagappan, N., Zimmermann, T., 2015. How practitioners perceive the relevance of software engineering research. In: Proceedings of the 2015 10Th Joint Meeting on Foundations of Software Engineering, pp. 415–425.

Ly, D., Overeem, M., Brinkkemper, S., Dalpiaz, F., 2025. The power of words in agile vs. waterfall development: written communication in hybrid software teams. J. Syst. Softw. 219, 112243.

Lyu, M.R., Ray, B., Roychoudhury, A., Tan, S.H., Thongtanunam, P., 2024. Automatic programming: large language models and beyond. ACM Trans. Softw. Eng. Method. .

Makrakis, G.M., Kolias, C., Kambourakis, G., Rieger, C., Benjamin, J., 2021. Industrial and critical infrastructure security: technical analysis of real-life security incidents. IEEE Access 9, 165295–165325.

Manikas, K., 2016. Revisiting software ecosystems research: a longitudinal literature study. J. Syst. Softw. 117, 84–103.

Manikas, K., Hansen, K.M., 2013. Software ecosystems–a systematic literature review. J. Syst. Softw. 86 (5), 1294–1306.

Monperrus, M., 2018. Automatic software repair: a bibliography. ACM Comput. Surv. (CSUR) 51 (1), 1–24.

Outão, J., Costa, L.A., Oliveira, R., Fontão, A., Constantinou, E., Santos, R., Serebrenik, A., 2025. Proprietary software ecosystems: what we already know and future perspectives. In: Proceedings of the 13Th ACM/IEEE International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems, pp. 1–10.

Romano, S., Scanniello, G., Marchetto, A., Giorgini, P., Guidetti, G., Converso, D., Viotti, S., 2024. Mood: mindfulness for software developers. In: Proceedings of the 18Th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, pp. 598–602.

Rugeviciute, A., Courboulay, V., Hilty, L.M., 2023. The research landscape of ICT for sustainability: harnessing digital technology for sustainable development. In: 2023 International Conference on ICT for Sustainability (ICT4S). IEEE, pp. 97–107.

Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case study research in software engineering. Empiric. Softw. Eng. 14, 131–164.

Runeson, P., Host, M., Rainer, A., Regnell, B., 2012. Case study research in software engineering: Guidelines and examples. John Wiley & Sons.

Russo, D., 2024. Navigating the complexity of generative ai adoption in software engineering. ACM Trans. Softw. Eng. Method. 33 (5), 1–50.

Santos, R.P., 2016. Managing and monitoring software ecosystem to support demand and solution analysis. Ph.D. thesis. Universidade Federal do Rio de Janeiro.

Sherje, N., 2024. Enhancing software development efficiency through AI-powered code generation. Res. J. Comput. Syst. Eng. 5 (1), 01–12.

Tembhekar, P., Devan, M., Jeyaraman, J., 2023. Role of genAI in automated code generation within devops practices: explore how generative AI. J. Knowl. Learn. Sci. Technol. ISSN: 2959–6386 (online) 2 (2), 500–512.

Tian, H., Lu, W., Li, T.O., Tang, X., Cheung, S.-C., Klein, J., Bissyandé, T.F., 2023. Is chatGPT the ultimate programming assistant–how far is it? arXiv preprint arXiv:2304.11938 .

Tian, Z., Qiu, R., Teng, Y., Sun, J., Chen, Y., Chen, L., 2025. Towards cost-efficient vulnerability detection with cross-modal adversarial reprogramming. J. Syst. Softw. , 112365.

Trinkenreich, B., Santos, G., Barcellos, M.P., Conte, T., 2019. Combining GQM + strategies and OKR-preliminary results from a participative case study in industry. In: International Conference on Product-Focused Software Process Improvement. Springer, pp. 103–111.

Vetro, A., 2012. Using automatic static analysis to identify technical debt. In: 2012 34Th International Conference on Software Engineering (ICSE). IEEE, pp. 1613–1615.

Vial, G., 2021. Understanding digital transformation: a review and a research agenda. Manag. Digit. Transform. , 13–66.

Wang, Y., Shen, S., Lim, B.Y., 2023. Reprompt: automatic prompt editing to refine ai-generative art towards precise expressions. In: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, pp. 1–29.

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C., 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint arXiv:2302.11382 .

Wohlin, C., Aurum, A., 2015. Towards a decision-making structure for selecting a research design in empirical software engineering. Empiric. Softw. Eng. 20 (6), 1427–1455.

Wohlin, C., Runeson, P., 2021. Guiding the selection of research methodology in industry–academia collaboration in software engineering. Inf. Softw. Technol. 140, 106678.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., et al., 2012. Experimentation in software engineering. Vol. 236. Springer.

Wu, H., Leung, S.-O., 2017. Can likert scales be treated as interval scales? a simulation study. J. Soc. Serv. Res. 43 (4), 527–532.

Wu, J., Wu, J., 2020. Security risks from vulnerabilities and backdoors. Cyberspace Mimic Defense: General. Robust Control Endogenous Secur. , 3–38.

Yin, R.K., 2005. Introducing the world of education: A case study reader. Sage.

Zamprogno, L., Hall, B., Holmes, R., Atlee, J.M., 2022. Dynamic human-in-the-loop assertion generation. IEEE Trans. Softw. Eng. 49 (4), 2337–2351.

Zubair, F., Al-Hitmi, M., Catal, C., 2024. The use of large language models for program repair. Comput. Stand. Interfac. , 103951.